
DebyeTools

Release 2.8.3

Javier Jofre

Oct 09, 2026

CONTENT:

1	Overview	3
1.1	How to cite	3
1.2	Calculate quality thermodynamic properties in a flexible and fast manner	3
1.3	Using <code>debyetools</code> through the GUI	5
1.4	Using <code>debyetools</code> as a Python library. Example: Al fcc	5
2	Installation	9
2.1	Source Code	9
2.2	Requirements	9
2.3	Installation using <code>pip</code>	9
3	GUI	11
3.1	General overview	11
3.2	Start window	12
3.3	Main window: parametrization	12
3.4	Results window	13
4	Examples	15
4.1	Al ₃ Li L1 ₂ thermodynamic properties	15
4.2	Thermodynamic properties with the <code>debyetools</code> interface	17
4.3	Genetic algorithm to fit Cp to experimental data	21
4.4	Simultaneous parameter adjusting to experimental heat capacity and thermal expansion at P = 0 and prediction of thermodynamic phase equilibria at high pressure	23
5	Thermodynamic Properties	27
5.1	Units	27
5.2	EOS parametrization	28
5.3	Poisson's ratio	33
5.4	Thermodynamic Properties	34
6	Input files, grids, constants	39
6.1	Loading VASP results	39
6.2	Temperature and pressure grids	39
6.3	Constants and unit conversions	41
7	Contributions to the free energy	43
7.1	Anharmonicity	43
7.2	Defects	50
7.3	Electronic Contribution	53
7.4	Excess contribution	59
7.5	Vibrational	61

8	FactSage compound database parametrization	63
8.1	Example	63
8.2	Source code	64
9	Pair Analysis Calculation	65
9.1	Example	65
9.2	Source code	66
10	Input file formats	67
10.1	Files and names	67
10.2	Reference structure (CONTCAR)	68
10.3	Energy-volume series (SUMMARY)	68
10.4	Electronic DOS (DOSCAR)	69
10.5	Elastic moduli (OUTCAR)	69
10.6	Direct inputs	70
	Python Module Index	71
	Index	73

`debyetools` is a set of tools written in [Python](#) for the calculation of thermodynamic and thermophysical properties. It's a library in The Python Package Index ([PyPI](#)). The software presented here is based in the Debye approximation of the quasiharmonic approximation (QHA) using the crystal internal energetics parametrized at ground-state, (go to [input file formats](#) to see how DFT calculations results can be used as inputs) to project the *thermodynamics properties* at high temperatures. How to cite `debyetools` and a first example are in the [overview](#). We present here how each contribution to the free energy are considered and a description of the architecture of the calculation engine and of the [GUI](#).

The [code](#) is freely available under the GNU Affero General Public License.

OVERVIEW

1.1 How to cite

If you use `debyetools` in a publication, please refer to the [source code](#). If you use the implemented method for the calculation of the thermodynamic properties, please cite the following publication:

Jofre, J., Gheribi, A. E., & Harvey, J.-P. Development of a flexible quasi-harmonic-based approach for fast generation of self-consistent thermodynamic properties used in computational thermochemistry. *Calphad* 83 (2023) 102624. doi: [10.1016/j.calphad.2023.102624](https://doi.org/10.1016/j.calphad.2023.102624).

```
@article{,
  author = {Javier Jofré and Aïmen E. Gheribi and Jean-Philippe Harvey},
  doi = {10.1016/j.calphad.2023.102624},
  issn = {03645916},
  journal = {Calphad},
  month = {12},
  pages = {102624},
  title = {Development of a flexible quasi-harmonic-based approach for fast
↪generation of self-consistent thermodynamic properties used in computational
↪thermochemistry},
  volume = {83},
  year = {2023},
}
```

1.2 Calculate quality thermodynamic properties in a flexible and fast manner

It's possible to couple the Debye model to other algorithms, to *fit experimental data* and in this way use data available to calculate other properties like thermal expansion, free energy, bulk modulus among many others.

The prediction of *thermodynamic phase equilibria at high pressure* can be performed by simultaneous parameter adjusting to experimental heat capacity and thermal expansion at $P = 0$.

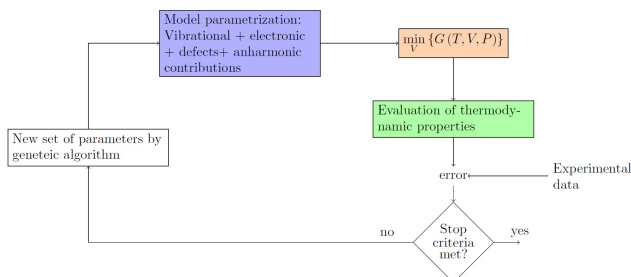
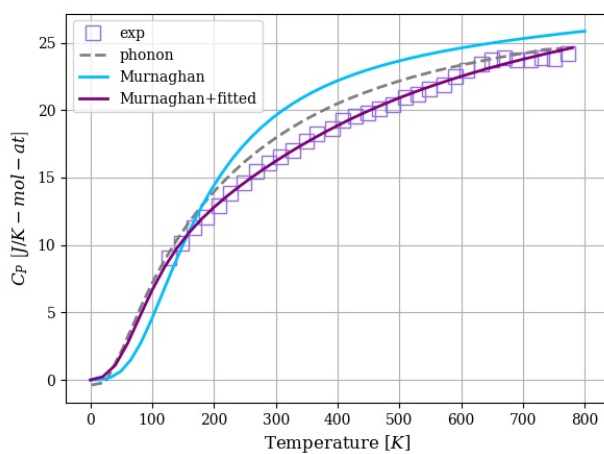
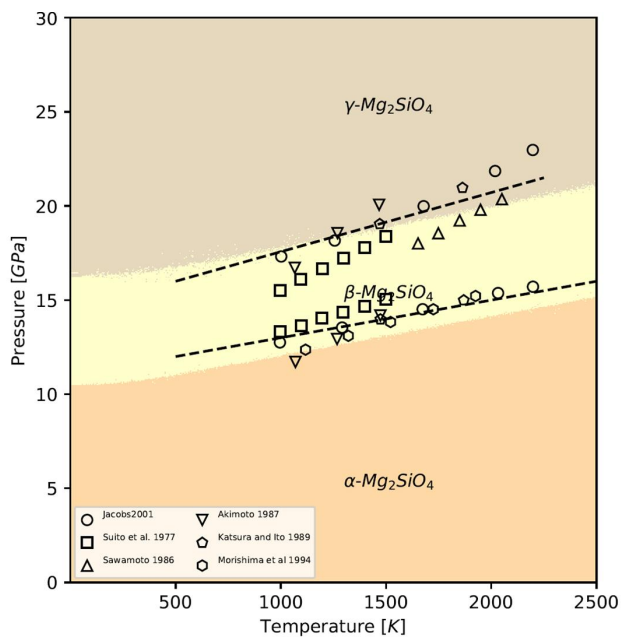


Fig. 1: debyetools coupled to a Genetic Algorithm.

Fig. 2: Heat capacity of LiFePO_4 calculated with debyetools and compared to other methods.Fig. 3: Phase diagram P versus T for the α , β and γ forms of Mg_2SiO_4 . Symbols are literature data for the phase stability regions boundaries.

1.3 Using debyetools through the GUI

debyetools is a [Python](#) library that also comes with a graphical user interface to help perform quick calculations without the need to code scripts.

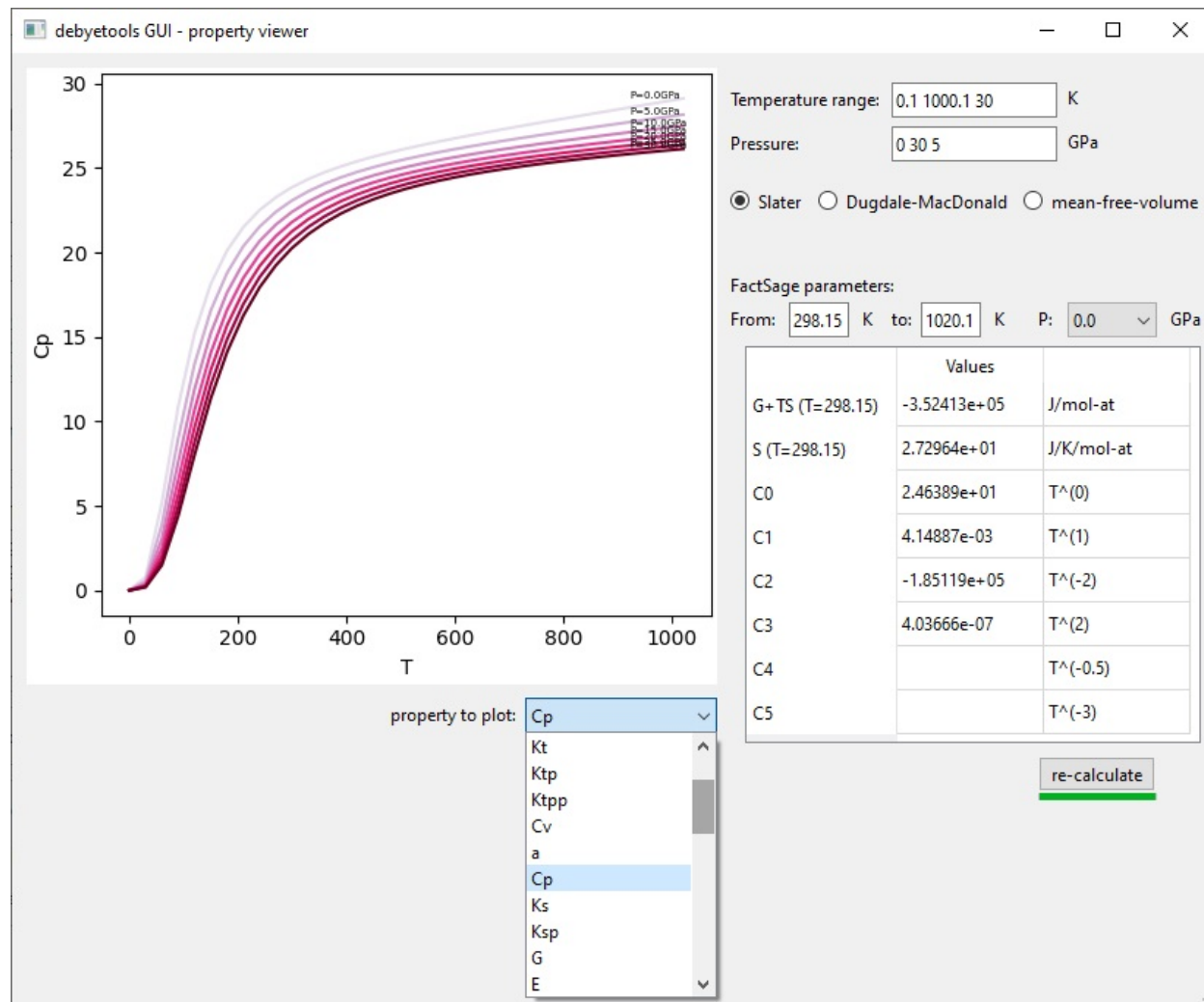


Fig. 4: debyetools property viewer.

1.4 Using debyetools as a Python library. Example: Al fcc

Using debyetools as a [Python](#) library adds versatility and expands its usability. The example uses the VASP results for Al fcc shipped with the package (debyetools/examples/Al_fcc). All inputs and outputs are SI per mole of atoms: V in m³/mol-at, energies in J/mol-at, pressures and moduli in Pa, mass in kg/mol-at.

Energy-volume data (units='J/mol'; the default 'eV/atom' returns Å³ and eV per atom) and EOS parametrization. The analytic EOS need no initial parameters; fitEOS returns the fitted parameters (E0, V0, K0, K0')

```
>>> import os
>>> import debyetools
```

(continues on next page)

(continued from previous page)

```
>>> from debyetools.aux_functions import load_V_E, load_doscar, load_EM, load_cell, \
↳ gen_Ts
>>> import debyetools.potentials as potentials
>>> folder = os.path.join(os.path.dirname(debyetools.__file__), 'examples', 'Al_fcc')
>>> V_data, E_data = load_V_E(folder + '/SUMMARY', folder + '/CONTCAR', units='J/mol')
>>> eos = potentials.BM()
>>> p_EOS = eos.fitEOS(V_data, E_data)
>>> print('E0 = %.4e J/mol-at, V0 = %.4e m3/mol-at, K0 = %.4e Pa, K0p = %.3f' % \
↳ tuple(p_EOS))
E0 = -3.6058e+05 J/mol-at, V0 = 9.9318e-06 m3/mol-at, K0 = 7.7683e+10 Pa, K0p = 4.580
```

The internal energy can also be described with an interatomic potential (Morse, EAM) built from the crystal structure; for Morse the parameters are (D, a, r0) per pair type:

```
>>> formula, cell, basis = load_cell(folder + '/CONTCAR')
>>> morse = potentials.MP(formula, cell, basis, 5, 3)
>>> p_MP = morse.fitEOS(V_data, E_data, initial_parameters=[0.35, 1, 3.2])
>>> print('D = %.4f, a = %.4f, r0 = %.4f; V0 = %.4e m3/mol-at' % (*p_MP, morse.V0))
D = 0.3517, a = 1.0114, r0 = 3.2359; V0 = 9.9608e-06 m3/mol-at
```

Electronic contribution: the total DOS at the Fermi level of each DOSCAR (one per volume, in the order of V_data) fitted to a cubic polynomial in V:

```
>>> from debyetools.electronic import fit_electronic
>>> tags = ['%02d' % i for i in range(1, 22)]
>>> E, N, Ef = load_doscar(folder + '/DOSCAR.EvV.', list_filetags=tags)
>>> p_electronic = fit_electronic(V_data, None, E, N, Ef)
>>> print(''.join('%.4e' % q for q in p_electronic))
4.2770e+00, -6.1244e+05, 3.4610e+09, 1.9514e+15
```

Poisson's ratio from the elastic moduli of a VASP IBRION = 6 OUTCAR:

```
>>> from debyetools.poisson import poisson_ratio
>>> nu = poisson_ratio(load_EM(folder + '/OUTCAR_elastic'))
>>> print('%.4f' % nu)
0.3370
```

Free energy minimization (equilibrium volume at each temperature, here at P = 0) with mono-vacancies (formation energy 8.46 k_B T_m, entropy 1.69 k_B) and no explicit anharmonicity:

```
>>> from debyetools.ndeb import nDeb
>>> m = 0.0269815385
>>> p_intanh, p_defects, p_anh = (0, 1), (8.46, 1.69, 933, 0.1), (0, 0, 0)
>>> ndeb = nDeb(nu, m, p_intanh, eos, p_electronic, p_defects, p_anh, mode='jjs1')
>>> T = gen_Ts(0.1, 1000, 11)
>>> T, V = ndeb.min_G(T, eos.V0, P=0)
```

Evaluation of the thermodynamic properties (a dictionary of arrays, e.g. 'Cp' in J/mol-at/K):

```
>>> tprops = ndeb.eval_props(T, V, P=0)
>>> for Ti, Vi, Cpi in zip(T, V, tprops['Cp']):
...     print('%7.2f K   V = %.4e   Cp = %.62f' % (Ti, Vi, Cpi))
    0.10 K   V = 1.0041e-05   Cp =    0.00
   100.09 K   V = 1.0054e-05   Cp =   12.26
   200.08 K   V = 1.0104e-05   Cp =   21.17
   298.15 K   V = 1.0170e-05   Cp =   24.19
```

(continues on next page)

(continued from previous page)

300.07 K	V = 1.0172e-05	Cp = 24.23
400.06 K	V = 1.0248e-05	Cp = 25.85
500.05 K	V = 1.0331e-05	Cp = 27.05
600.04 K	V = 1.0420e-05	Cp = 28.19
700.03 K	V = 1.0518e-05	Cp = 29.47
800.02 K	V = 1.0624e-05	Cp = 31.12
900.01 K	V = 1.0743e-05	Cp = 33.44
1000.00 K	V = 1.0879e-05	Cp = 36.79

FactSage compound-database parameters (heat capacity $C_p = c_0 + c_1 T + c_2 T^{-2} + c_3 T^2 + c_4 T^{-1/2} + c_5 T^{-3}$; $c_5 = 0$ unless `cp_T3=True`):

```
>>> from debyetools.fs_compound_db import fit_FS
>>> FS_db_params = fit_FS(tprops, 298.15, 1000)
>>> print(', '.join('%3e' % c for c in FS_db_params['Cp']))
1.803e+02, -1.429e-01, 1.564e+06, 7.288e-05, -2.376e+03, 0.000e+00
```


INSTALLATION

2.1 Source Code

All the code is in `debyetools` repository on [GitHub](#).

2.2 Requirements

- [Python](#) 3.10, 3.11 or 3.12
- [NumPy](#) 1.26 or 2.x (base N-dimensional array package)
- [SciPy](#) 1.11 or newer (fundamental algorithms for scientific computing in Python)

For the graphical interface and plotting:

- [Matplotlib](#) (plots in the GUI and in the examples)
- [PySide6](#) (the GUI; PySide6 is the official Python module from the Qt for Python project)

Running the tests needs [pytest](#) (`python -m pytest` from the repository folder); the GUI tests run headless (`QT_QPA_PLATFORM=offscreen`) and are skipped when PySide6 cannot be loaded.

2.3 Installation using pip

The simplest way to install `debyetools` is to use [pip](#) which will automatically get the source code from [PyPI](#):

```
$ pip install --upgrade debyetools
```


Table of contents

- *General overview*
 - *How to launch it*
- *Start window*
 - *Crystal structure (Morse, EAM)*
- *Main window: parametrization*
 - *EOS fit*
 - *Poisson's ratio and elastic properties*
 - *Electronic contribution*
- *Results window*
 - *Reference energies*

3.1 General overview

The graphical interface parametrizes the free energy, runs the calculation and shows the results without writing scripts. It uses the same calculation engine as the library (`debyetools.ndeb.nDeb`); all values are per mole of atoms unless stated otherwise. Most input files are VASP results (see *input file formats*): SUMMARY and CONTCAR for the energy curve, DOSCAR files for the electronic contribution and an IBRION = 6 OUTCAR for the elastic constants. Example input files are in `debyetools/examples` and in the *examples input files* of the repository.

3.1.1 How to launch it

From the repository main folder:

```
$ python dtgui.py
```

or inside Python:

```
>>> from debyetools.tpropsgui.gui import dtgui
>>> dtgui()
```

Messages (fit restarts, truncated temperature ranges, missing references, ...) are shown in message boxes; errors are also printed to the console.

3.2 Start window

- **Compound**: click the field to open the periodic table and set the number of atoms of each element in the formula unit.
- **mass [kg/mol-at]**: filled from the formula, as the arithmetic (default) or the logarithmic mean of the atomic masses (radio buttons next to the field; the logarithmic mean is recommended by Lu et al. (2007) for large mass differences).
- **Specify structure (for interatomic potential)**: tick it to describe the internal energy with the Morse or EAM potential; the crystal dialog opens with **Next**. Without it, the analytic EOS are offered (Birch-Murnaghan, Rose-Vinet, Mie-Gruneisen, TB-SMA, Murnaghan, Poirier-Tarantola).

3.2.1 Crystal structure (Morse, EAM)

Enter the cell (Å), the fractional basis with the element of each atom, the cut-off radius (Å) and the number of neighbour shells, then **Update**: the cell is drawn, the pair analysis is listed (distances and number of pairs per pair type) and the default initial parameters of the potential are written (Morse: 3 per pair type; EAM: 6 per pair type and 4 per element type). **Next** opens the main window with Morse and EAM in the EOS list.

3.3 Main window: parametrization

- **EOS params**: the parameters of the selected EOS, typed or obtained with **fit parameters**. Selecting another EOS writes its default start (for Morse and EAM, the one for the crystal's pair types).
- **Poisson's ratio**: typed or obtained with **calculate Poisson's ratio**.
- **electronic contribution** ($q_0 \dots q_3$ of $N(E_F)(V)$), **mono-vacancies** (Evac00, Svac00, Tm, a), **intrinsic anh.** (a_0 , m_0), **explicit anh.** (s_0 , s_1 , s_2) and **excess polynomial** ($xs_0 \dots xs_5$): each term is used only when its box is ticked; missing trailing coefficients are zero (see *contributions* for the definitions and units).
- **T** and **P**: `start end step` in K and GPa (a single value is allowed); **run >** opens the results window.

3.3.1 EOS fit

Paste the energy curve as two columns (V, E) or load it with **load E(V) ...** from a SUMMARY and a POSCAR/CONTCAR file, choose the units (**eV and Å³ (per at)** or **J and m³ (per mol-at)**) and **fit**. The analytic EOS need no initial parameters; if a fit is not acceptable, other starting points are tried, and when none passes the dialog asks whether to keep the best attempt. **plot E(V)** compares the fit with the data; **Save and Close** copies the parameters to the main window. EAM fits of compounds can take a minute or more, during which the window does not respond.

3.3.2 Poisson's ratio and elastic properties

Paste the stiffness tensor in GPa (Voigt order XX, YY, ZZ, YZ, ZX, XY) or load it with **Load tensor...** from an OUTCAR (converted from kBar and from the VASP order; the POTCAR names in the OUTCAR are also kept for the reference energies). **Calculate** gives the Voigt, Reuss and Hill bulk and shear moduli, the universal anisotropy index and the Poisson's ratio. **More...** opens a report (eigenvalues of the stiffness matrix, averages, minimum and maximum of the Young's modulus, linear compressibility, shear modulus and Poisson's ratio with their directions) and **Plots** draws them in the xy, xz and yz planes (negative values in green, as magnitudes).

3.3.3 Electronic contribution

calculate parameters next to the electronic contribution opens a dialog to select the DOSCAR files (one per volume, in any order: each file is paired with the volume written in it) and fits $N(E_F)(V)$. If $E(V)$ data were entered in the EOS fit, the volumes are cross-checked.

3.4 Results window

The equilibrium volume is calculated at each temperature and pressure and the properties are evaluated. The property to plot is chosen from a list, with one curve per pressure; right-clicking the figure copies the data to the clipboard. The temperature and pressure ranges and the Debye-temperature model (**Slater, Dugdale-MacDonald, mean-free-volume**) can be changed and the calculation repeated with **re-calculate**. When no stable volume exists beyond some temperature, the calculation stops there and a message says so.

The table shows, for the selected pressure, per mol-atom: $G + TS$ and S at 298.15 K and the FactSage heat-capacity coefficients $C_0 \dots C_5$ fitted between **From** and **to** (the T^{-3} term only when its box is ticked), and, per formula unit, the values exported for the FactSage Compound module:

- **H298**: the enthalpy of formation $\Delta H_{298} = H_{\text{compound}}(298.15) - \sum n_i H_i(298.15)$. The element references H_i are the results of pure-element runs made in the same session (same POTCAR and settings, $P = 0$) or values entered in **Reference energies...**; without them the static formation energy $E_0(V_0) - \sum n_i E_i$ is exported and a message says so.
- **S298**: the entropy at 298.15 K.

export parameters writes the formula, H298, S298, the Cp coefficients and the temperature window to the file `export_dtoutput4cmpnd` (the same content is written to `dtoutput4cmpnd` after every run), in the working folder.

3.4.1 Reference energies

Reference energies... lists, for each element of the formula, the POTCAR (read from an OUTCAR, or assumed to be the element symbol), the static reference energy E_{ref} (eV/atom; built-in values for many PAW_PBE POTCARs) and the reference enthalpy $H_{298 \text{ ref}}$ (J/mol-atom). Values can be edited for the session, saved to and loaded from a CSV file (**Save table...**, **Load table...**; file format and a script for placeholder values in `debyetools/tpropsgui/reference_tables`), and the POTCAR names can be read from an OUTCAR. The references must come from calculations with the same settings as the compound, and E_{ref} is the energy at zero smearing (VASP's E_0 , the value `load_V_E` reads; the built-in values follow this convention). Placeholder values from the Materials Project are in `reference_tables/mp_placeholders_PBE.csv`; for H, N, O, F and Cl they are uncorrected PBE energies (O₂ overbinding: oxide formation energies too negative by about 0.7 eV per O atom; see the README in that folder).

EXAMPLES

Table of contents

- *Al₃Li L1₂ thermodynamic properties*
- *Thermodynamic properties with the `debyetools` interface*
- *Genetic algorithm to fit C_p to experimental data*
- *Simultaneous parameter adjusting to experimental heat capacity and thermal expansion at $P = 0$ and prediction of thermodynamic phase equilibria at high pressure*

4.1 Al₃Li L1₂ thermodynamic properties

To calculate the thermodynamic properties of an element or compound, we first parametrize the function that describes its internal energy. Here the third-order Birch-Murnaghan equation of state (BM) is fitted to the DFT energy-volume data loaded with `load_V_E` (the VASP energies extrapolated to zero smearing, $E_0=$; see *Input file formats*). The module `potentials` holds all the implemented EOS; `fitEOS` fits the parameters to (volume, energy) data and returns them (E_0 , V_0 , K_0 , K_0'). All quantities are SI per mole of atoms. The VASP results for Al₃Li L1₂ are shipped with the package:

```
>>> import os
>>> import debyetools
>>> folder = os.path.join(os.path.dirname(debyetools.__file__), 'examples', 'Al3Li_L12
↪')
```

Loading the energy curve and fitting the Birch-Murnaghan EOS:

```
>>> from debyetools.aux_functions import load_V_E
>>> import debyetools.potentials as potentials
>>> V_DFT, E_DFT = load_V_E(folder + '/SUMMARY', folder + '/CONTCAR', units='J/mol')
>>> eos_BM = potentials.BM()
>>> p_EOS = eos_BM.fitEOS(V_DFT, E_DFT)
>>> print(', '.join('%4e' % p for p in p_EOS))
-3.2645e+05, 9.8233e-06, 6.3223e+10, 4.3026e+00
```

The electronic contribution uses the density of states at the Fermi level. The DOSCAR of each volume of `V_DFT` (same order) is read with `load_doscar` (total DOS, spins summed, per atom) and $N(E_F)$ is fitted to a cubic polynomial in V with `fit_electronic` (linear least squares; the second argument is not used):

```
>>> from debyetools.aux_functions import load_doscar
>>> from debyetools.electronic import fit_electronic
```

(continues on next page)

(continued from previous page)

```
>>> list_filetags = ['%02d' % i for i in range(1, 22)]
>>> E, N, Ef = load_doscar(folder + '/DOSCAR.EvV.', list_filetags=list_filetags)
>>> p_electronic = fit_electronic(V_DFT, None, E, N, Ef)
>>> print(', '.join('%4e' % q for q in p_electronic))
-1.0588e+00, 4.3832e+05, -4.9644e+10, 1.8880e+15
```

The Poisson's ratio follows from the elastic moduli matrix of the VASP IBRION = 6 OUTCAR, read with load_EM:

```
>>> from debyetools.aux_functions import load_EM
>>> from debyetools.poisson import poisson_ratio
>>> nu = poisson_ratio(load_EM(folder + '/OUTCAR_elastic'))
>>> print('%4f' % nu)
0.2294
```

The mass is the mean atomic mass of Al_3Li , $(3 m_{\text{Al}} + m_{\text{Li}})/4$ in kg/mol-at. For this example the vacancies, the anharmonic terms and the excess term are switched off:

```
>>> m = (3 * 26.9815385 + 6.94) / 4 / 1000
>>> print('%6f' % m)
0.021971
>>> Tmelting = 933
>>> p_defects = 1e10, 0, Tmelting, 0.1
>>> p_intanh = 0, 1
>>> p_anh = 0, 0, 0
```

The temperature dependence of the equilibrium volume is calculated by minimizing the Gibbs energy, here at $P = 0$. We first create an nDeb object and define the temperatures (gen_Ts adds 298.15 K); the minimization is done by nDeb.min_G, starting from the EOS V0:

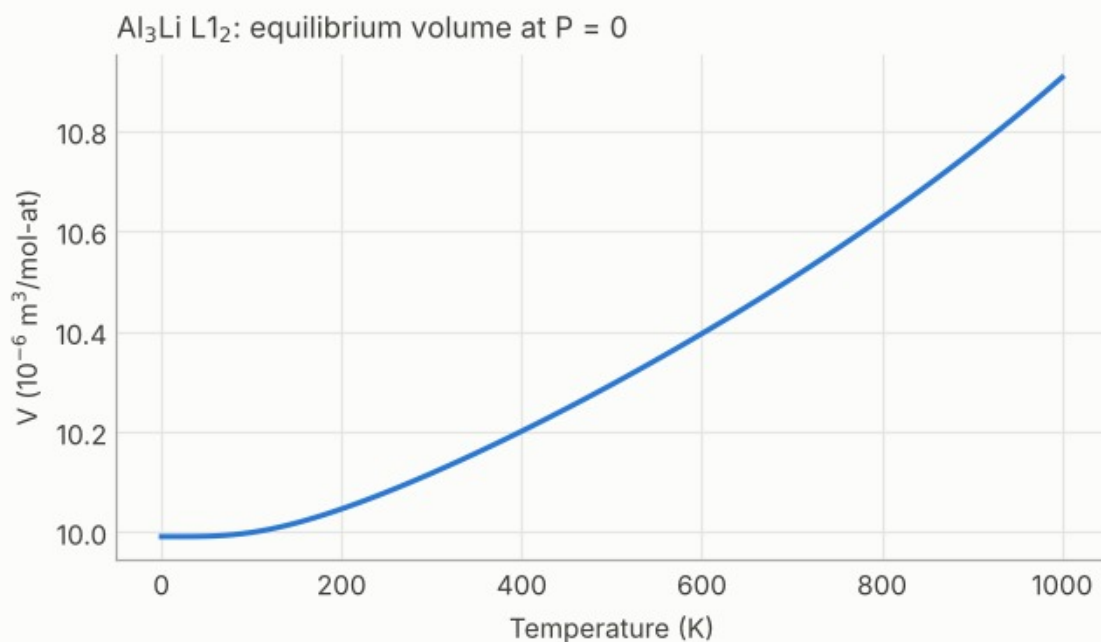
```
>>> from debyetools.ndeb import nDeb
>>> from debyetools.aux_functions import gen_Ts
>>> ndeb_BM = nDeb(nu, m, p_intanh, eos_BM, p_electronic, p_defects, p_anh)
>>> T = gen_Ts(0.1, 1000, 11)
>>> T, V = ndeb_BM.min_G(T, eos_BM.V0, P=0)
```

The thermodynamic properties are obtained by evaluating the thermodynamic functions with nDeb.eval_props, which returns a dictionary of arrays:

```
>>> tprops_dict = ndeb_BM.eval_props(T, V, P=0)
>>> for Ti, Vi, Cpi in zip(T, V, tprops_dict['Cp']):
...     print('%7.2f K  V = %4e  Cp = %6.2f' % (Ti, Vi, Cpi))
0.10 K  V = 9.9903e-06  Cp = 0.00
100.09 K  V = 9.9986e-06  Cp = 8.13
200.08 K  V = 1.0046e-05  Cp = 18.41
298.15 K  V = 1.0115e-05  Cp = 22.61
300.07 K  V = 1.0117e-05  Cp = 22.66
400.06 K  V = 1.0201e-05  Cp = 24.85
500.05 K  V = 1.0294e-05  Cp = 26.34
600.04 K  V = 1.0395e-05  Cp = 27.60
700.03 K  V = 1.0506e-05  Cp = 28.82
800.02 K  V = 1.0626e-05  Cp = 30.13
900.01 K  V = 1.0759e-05  Cp = 31.65
1000.00 K  V = 1.0907e-05  Cp = 33.53
```

To plot them (the figures use 101 temperatures, gen_Ts(0.1, 1000, 101)):

```
from matplotlib import pyplot as plt
plt.plot(T, V)
plt.xlabel('Temperature (K)')
plt.ylabel('V (m$^3$/mol-at)')
plt.show()
```



The FactSage Cp polynomial is fitted to the calculated properties between 298.15 and 1000 K (fit_FS returns a dictionary; see *FactSage compound database parametrization*):

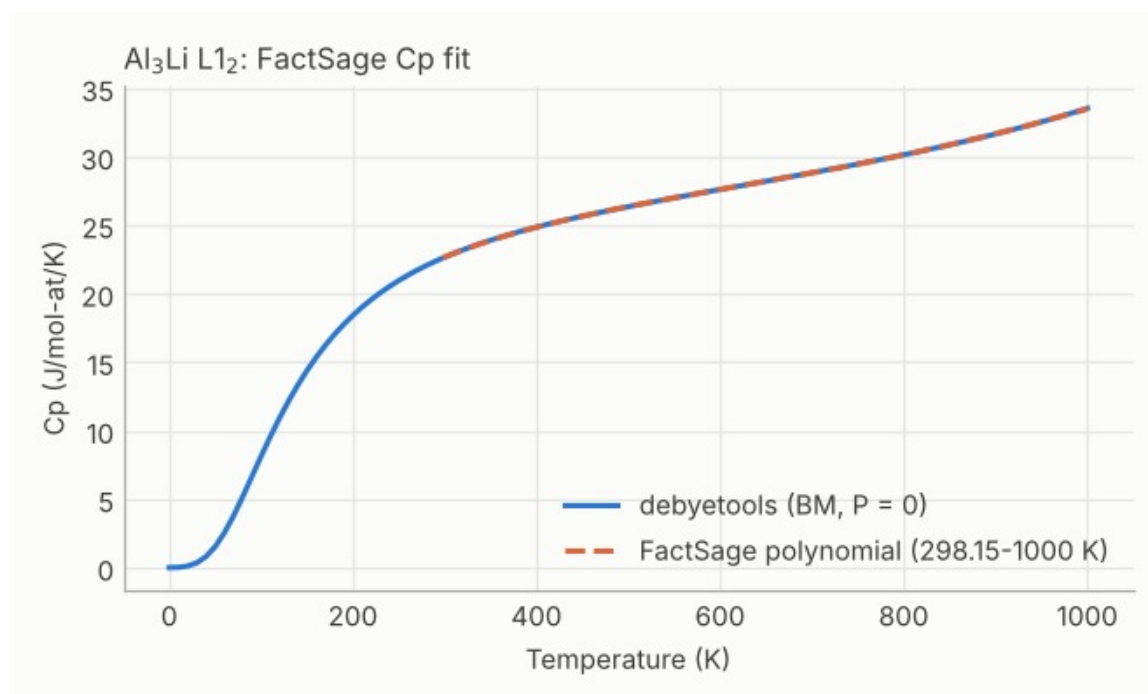
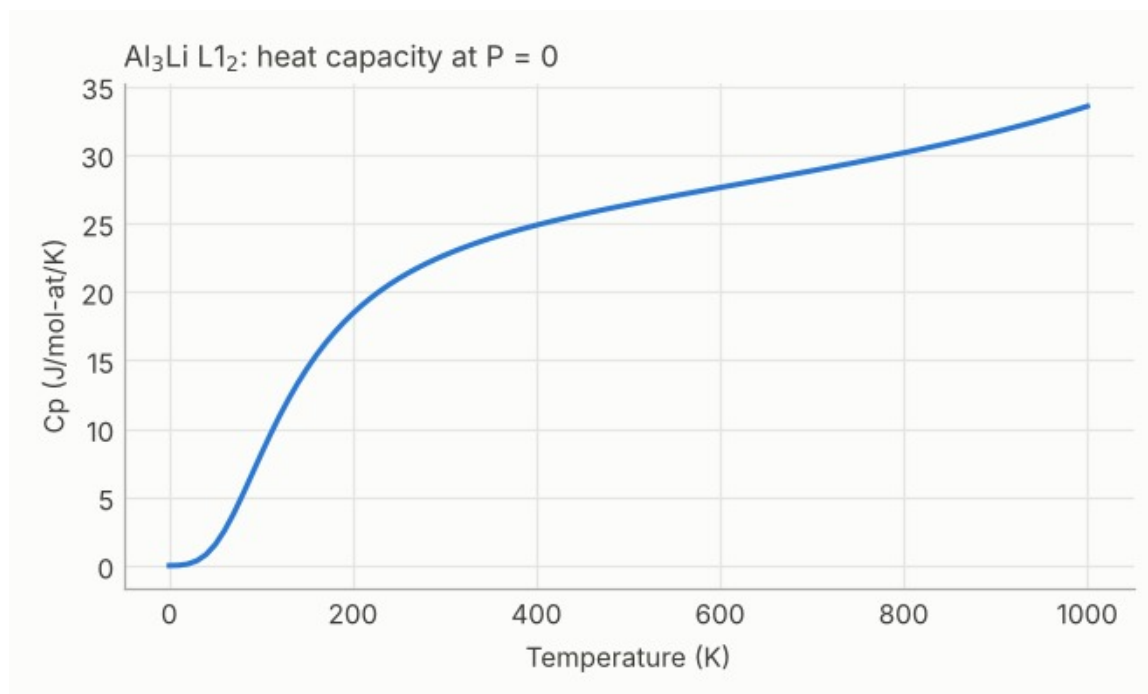
```
>>> from debyetools.fs_compound_db import fit_FS
>>> FS_db_params = fit_FS(tprops_dict, 298.15, 1000)
>>> for key, values in FS_db_params.items():
...     print(key, ', '.join('%0.3e' % v for v in values))
Cp 7.748e+01, -4.441e-02, 3.195e+05, 2.609e-05, -8.210e+02, 0.000e+00
a -9.658e-05, 1.783e-07, 7.247e-02, -1.088e+01
1/Ks 1.598e-11, 3.446e-15, -8.719e-19, 2.328e-21
Ksp 4.479e+00, 1.188e-03
```

The fitted polynomial against the calculated heat capacity:

4.2 Thermodynamic properties with the debyetools interface

The same calculations as the previous example can be carried out with the debyetools *GUI*.

The results are plotted in the results window that opens with **run >**; here the number of temperatures was increased from the default to show smoother curves.



debyetools GUI

plot

mass: kg/mol-at

E(V):

#V	E
1.201468E+01	-3.210751E+00
1.241964E+01	-3.326448E+00
1.283359E+01	-3.424838E+00

units:

☒ eV and A³ (per at)

☐ J and m³ (per mol-at)

EOS params: Birch-Murnaghan

Stiffness Tensor (GPa)

96.97	68.53	68.53	0	0	0
68.53	96.97	68.53	0	0	0
68.53	68.53	96.97	0	0	0
0.00	0	0	45.32	0	0
0.00	0	0	0	45.32	0
0.00	0	0	0	0	45.32

B (Voigt): GPa S (Voigt): GPa

B (Reuss): GPa S (Reuss): GPa

B (Voigt-Reuss-Hill): GPa S (Voigt-Reuss-Hill): GPa

Universal anisotropy: GPa Poisson's ratio: GPa

☒ electronic contribution:

☐ mono-vacancies:

☐ explicit anh.:

☐ excess:

Fig. 1: debyetools main interface

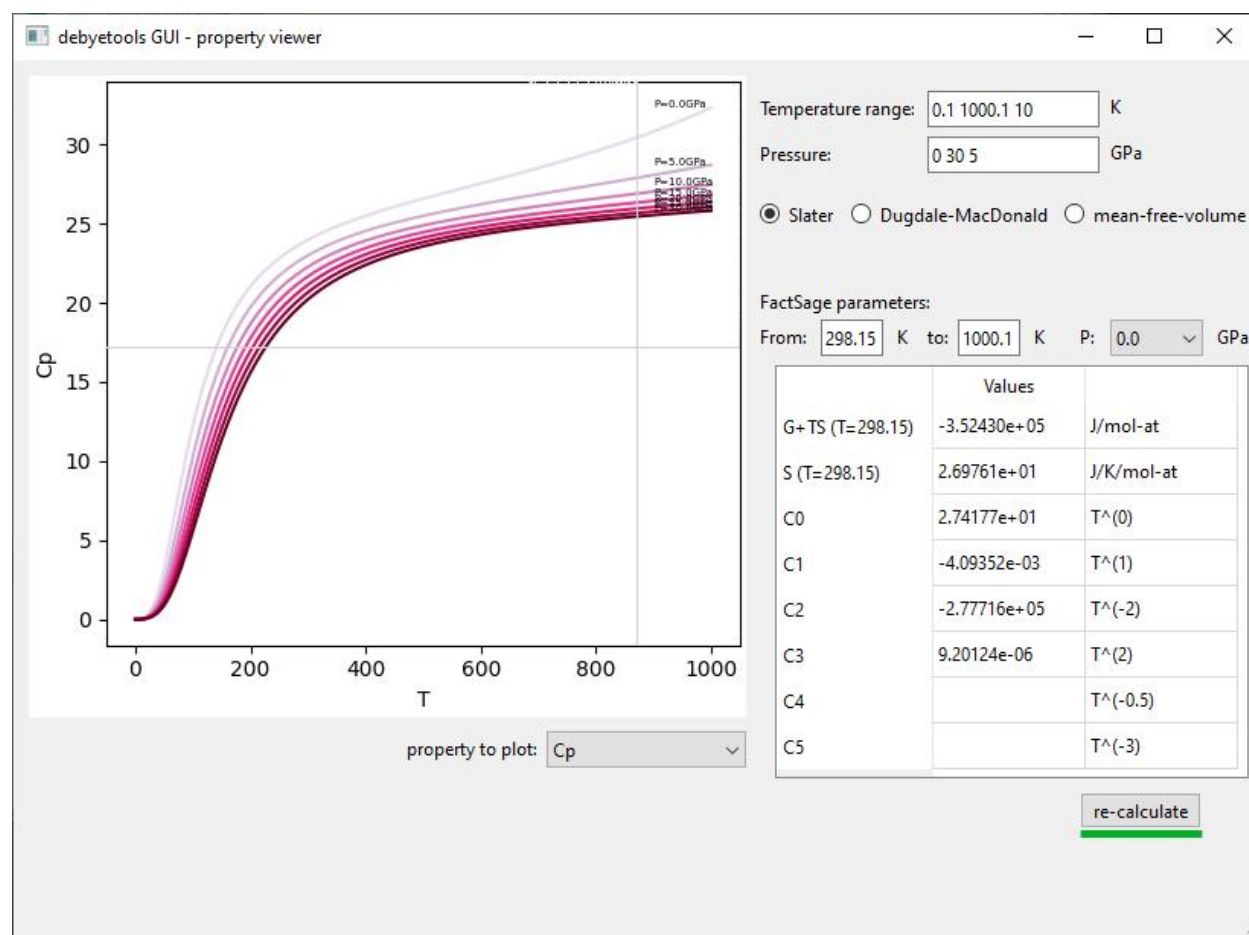


Fig. 2: debyetools viewer window

4.3 Genetic algorithm to fit Cp to experimental data

To show how flexible `debyetools` is, we show next a way to fit a thermodynamic property like the heat capacity to experimental data using a genetic algorithm.

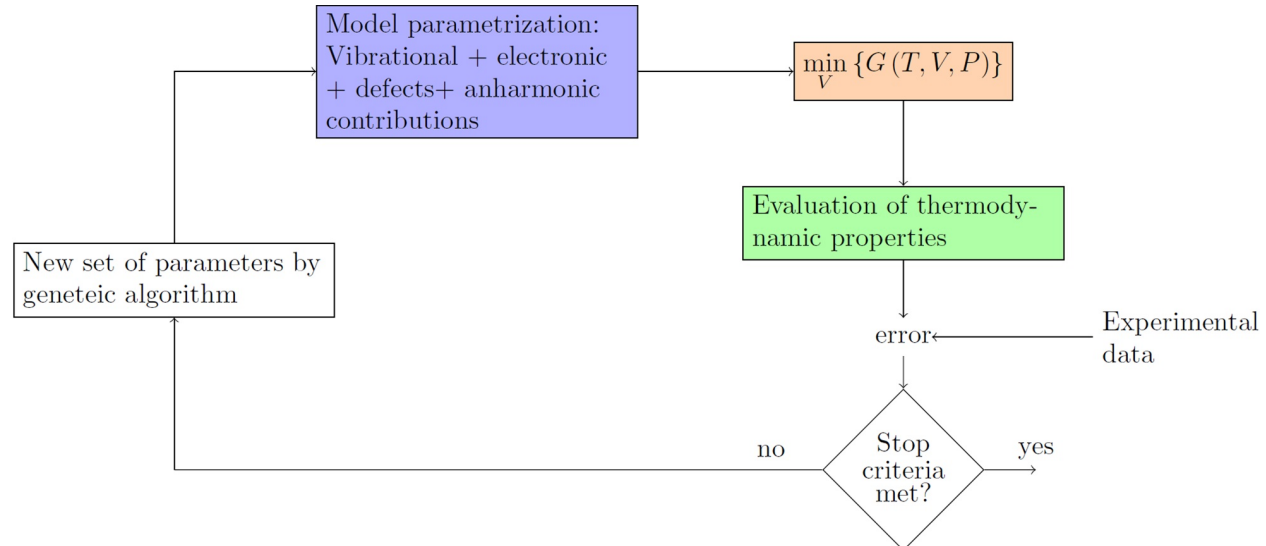


Fig. 3: Schematics of the fitting of the heat capacity to experimental data.

The example below fits the Murnaghan EOS parameters V_0 , K_0 , K_0' , the Poisson's ratio, the intrinsic and explicit anharmonicity and the vacancy parameters of LiFePO_4 to its experimental heat capacity, with the Dugdale-MacDonald Debye temperature (`mode='jjdm'`). It is shown as code to adapt rather than to run as is (a genetic algorithm evaluates the model many times).

First we set the initial values and the experimental data (T in K, Cp in J/mol-at/K):

```
import numpy as np
import numpy.random as rnd
import debyetools.potentials as potentials
from debyetools.ndeb import nDeb

eos_MU = potentials.MU()
params_Murnaghan = [-6.745375544e+05, 6.405559904e-06, 1.555283892e+11, 4.
    ↪ 095209375e+00]
E0, V0, K0, K0p = params_Murnaghan
nu = 0.2747222272342077
a0, m0 = 0, 1
s0, s1, s2 = 0, 0, 0
edef, sdef = 20, 0
V0_ini, K0_ini, K0p_ini, nu_ini = V0, K0, K0p, nu # kept for the final plot
T = np.array([126.9565217, 147.826087, 167.826087, 186.9565217, 207.826087, 226.9565217,
    ↪ 248.6956522, 267.826087, 288.6956522, 306.9565217, 326.9565217, 349.5652174, 366.9565217,
    ↪ 391.3043478, 408.6956522, 428.6956522, 449.5652174, 467.826087, 488.6956522, 510.4347826,
    ↪ 530.4347826, 548.6956522, 571.3043478, 590.4347826, 608.6956522, 633.0434783, 649.5652174,
    ↪ 670.4347826, 689.5652174, 711.3043478, 730.4347826, 750.4347826, 772.173913])
C_exp = np.array([9.049180328, 10.14519906, 11.29742389, 12.05620609, 12.92740047, 13.
    ↪ 82669789, 14.61358314, 15.45667447, 16.07494145, 16.55269321, 17.00234192, 17.73302108, 18.
    ↪ 21077283, 18.60421546, 19.25058548, 19.53161593, 19.78454333, 20.12177986, 20.4028103, 20.
    ↪ 90866511, 21.18969555, 21.52693208, 21.89227166, 22.4824356, 22.96018735, 23.40983607, 23.
    ↪ 69086651, 23.88758782, 23.71896956, 23.7470726, 23.85948478, 23.83138173, 24.19672131])
```

The function to evaluate, the heat capacity at the temperatures T (an array):

```
def Cp_LiFePO4(T, params):
    V0, K0, K0p, nu, a0, m0, s0, s1, s2, edef, sdef = params
    p_intanh = a0, m0
    p_anh = s0, s1, s2

    # EOS with the trial parameters (used as given, fit=False)
    eos_MU.fitEOS([V0], [E0], initial_parameters=[E0, V0, K0, K0p], fit=False)

    p_electronic = [0, 0, 0, 0]
    Tmelting = 800
    p_defects = edef, sdef, Tmelting, 0.1
    m = 0.02253677142857143

    ndeb_MU = nDeb(nu, m, p_intanh, eos_MU, p_electronic, p_defects, p_anh, mode='jjdm
    ↪')
    T_ok, V = ndeb_MU.min_G(T, V0, P=0)
    if len(T_ok) < len(T):
        raise ValueError('no stable volume above %.1f K' % T_ok[-1]) # min_G stops
    ↪at the last stable T
    return ndeb_MU.eval_props(T_ok, V, P=0) ['Cp']
```

The genetic algorithm: mutate varies the parameters, mate combines two parent sets, evaluate is the error against the experiment (a failed evaluation counts as a large error) and select_bests keeps the best sets:

```
def mutate(params, n_children, mrate, mvar):
    res = []
    for i in range(n_children):
        new_params = []
        for pi, mvars in zip(params, mvar):
            if rnd.randint(0, 100) / 100. <= mrate:
                step = mvars[1] / 10
                lst1 = np.arange(mvars[0] - mvars[1], mvars[0] + mvars[1] + step,
    ↪step)
                new_params.append(lst1[rnd.randint(0, len(lst1))])
            else:
                new_params.append(pi)
        res.append(new_params)
    return res

def evaluate(fc, T, pi, yexp):
    try:
        return np.sqrt(np.sum((fc(T, pi) / T - yexp / T) ** 2))
    except Exception:
        return 1e10

def select_bests(fn, T, params, ngen, yexp):
    errs = np.array([evaluate(fn, T, pi, yexp) for pi in params])
    best = np.argsort(errs)[:ngen]
    return [params[j] for j in best], [errs[j] for j in best]

def mate(params, ngen, mvar):
    res = [params[0], params[1]]
    for i in range(int(max(2, ngen - 2) / 2)):
        cutsite = rnd.randint(0, len(params[0]))
        res.append(mutate(params[0][:cutsite] + params[1][cutsite:], 1, 0.5, mvar)[0])
        res.append(mutate(params[1][:cutsite] + params[0][cutsite:], 1, 0.5, mvar)[0])
```

(continues on next page)

(continued from previous page)

```
return res
```

The iterations (1) mate the parents, (2) evaluate the children and (3) keep the best two as the new parents, until the best error has not changed for 20 generations or `max_iter` is reached:

```
mvar = [(V0, V0*0.01), (K0, K0*0.05), (K0p, K0p*0.01), (nu, nu*0.01), (a0, 5e-6), (m0,
↪ 5e-3),
        (s0, 5e-5), (s1, 5e-5), (s2, 5e-5), (edef, 0.5), (sdef, 0.1)]
parents_params = mutate([V0, K0, K0p, nu, a0, m0, s0, s1, s2, edef, sdef], n_
↪ children=2, mrate=0.7, mvar=mvar)
ix, max_iter, counter_change, errs_old = 0, 500, 0, 1
while ix <= max_iter:
    children_params = mate(parents_params, 10, mvar)
    parents_params, errs_new = select_bests(Cp_LiFePO4, T, children_params, 2, C_exp)
    V0, K0, K0p, nu, a0, m0, s0, s1, s2, edef, sdef = parents_params[0]
    mvar = [(V0, V0*0.05), (K0, K0*0.05), (K0p, K0p*0.05), (nu, nu*0.05), (a0, 5e-6),
↪ (m0, 5e-3),
            (s0, 5e-5), (s1, 5e-5), (s2, 5e-5), (edef, 0.5), (sdef, 0.1)]
    counter_change = counter_change + 1 if errs_old == errs_new[0] else 0
    errs_old = errs_new[0]
    ix += 1
    if counter_change >= 20:
        break
best_params = parents_params[0]
```

The result can be plotted with matplotlib against the experiment (and, here, a phonon calculation):

```
from matplotlib import pyplot as plt

T_calc = np.linspace(0.1, 800, 51)
plt.plot(T, C_exp, 'o', label='exp')
plt.plot(T_calc, Cp_LiFePO4(T_calc, [V0_ini, K0_ini, K0p_ini, nu_ini, 0, 1, 0, 0, 0,
↪ 20, 0]), label='Murnaghan')
plt.plot(T_calc, Cp_LiFePO4(T_calc, best_params), label='Murnaghan + fitted')
plt.xlabel('Temperature (K)')
plt.ylabel('Cp (J/mol-at/K)')
plt.legend()
plt.show()
```

The resulting figure is:

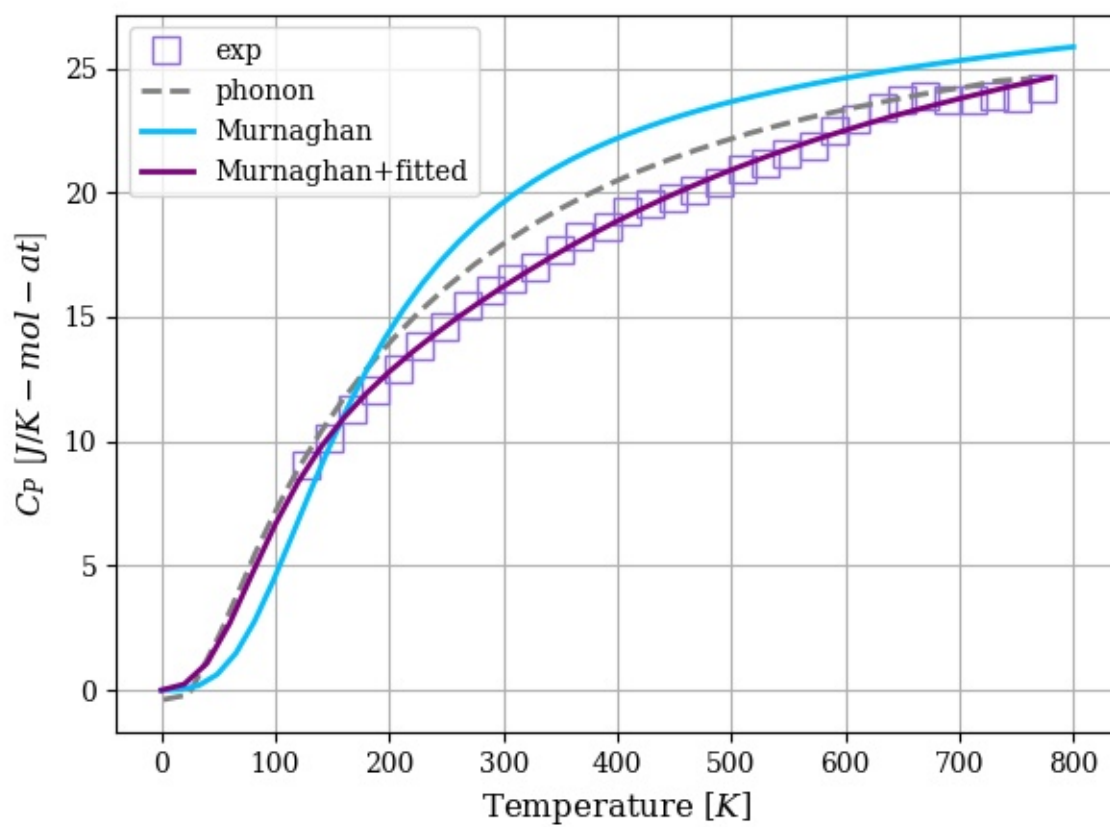
4.4 Simultaneous parameter adjusting to experimental heat capacity and thermal expansion at $P = 0$ and prediction of thermodynamic phase equilibria at high pressure

Similarly to the previous example, a genetic algorithm was used to adjust the model parameters to experimental data. The compound studied was Mg_2SiO_4 in the α , β and γ phases (forsterite, wadsleyite and ringwoodite) with structures Pnma, Imma and Fd-3m, respectively, for temperatures from 0 to 2500 K and pressures from 0 to 30 GPa. The isobaric heat capacity and the thermal expansion were fitted simultaneously at $P = 0$, so the objective function returns both (same structure as `Cp_LiFePO4`, with the parameters of each phase):

```
def Cp_a_Mg2SiO4(T, params):
    V0, K0, K0p, nu, a0, m0, s0, s1, s2, edef, sdef = params
```

(continues on next page)

4.4. Simultaneous parameter adjusting to experimental heat capacity and thermal expansion at $P = 0$ and prediction of thermodynamic phase equilibria at high pressure

Fig. 4: LiFePO_4 heat capacity.

(continued from previous page)

```

eos_MU.fitEOS([V0], [E0], initial_parameters=[E0, V0, K0, K0p], fit=False)
ndeb_MU = nDeb(nu, m, (a0, m0), eos_MU, [0, 0, 0, 0], (edef, sdef, Tmelting, 0.1),
→ (s0, s1, s2),
        mode='jjdm')
T_ok, V = ndeb_MU.min_G(T, V0, P=0)
if len(T_ok) < len(T):
    raise ValueError('no stable volume above %.1f K' % T_ok[-1])
tprops_dict = ndeb_MU.eval_props(T_ok, V, P=0)
return tprops_dict['a'], tprops_dict['Cp']

```

The genetic algorithm is the same as in the previous example except for the evaluation function, which now takes target data for both the thermal expansion and the heat capacity (T_{set1} , T_{set2} : their temperatures):

```

def evaluate(fc, T_set1, T_set2, pi, yexp, yexp2):
    try:
        alpha = fc(T_set1, pi)[0]
        Cp = fc(T_set2, pi)[1]
        err1 = np.sqrt(np.sum(((alpha - yexp) / yexp) ** 2)) / len(T_set1)
        err2 = np.sqrt(np.sum(((Cp - yexp2) / yexp2) ** 2)) / len(T_set2)
        return err1 + err2
    except Exception:
        return 1e10

```

Once the optimal parameters of the three phases are obtained (one nDeb object per phase: ndeb_alpha, ndeb_beta, ndeb_gamma), the thermodynamic properties are calculated as functions of temperature and pressure; for each phase:

```

from debyetools.aux_functions import gen_Ts, gen_Ps

Ts = gen_Ts(0.1, 2500, 100)
Ps = gen_Ps(0, 30e9, 31)

def props_TP(ndeb, V0):
    tprops = []
    for P in Ps:
        T, V = ndeb.min_G(Ts, V0, P=P)      # equilibrium volume at each T (stops at_
→the last stable T)
        tprops.append(ndeb.eval_props(T, V, P=P))
    return tprops

tprops_alpha = props_TP(ndeb_alpha, V0_alpha)
tprops_beta = props_TP(ndeb_beta, V0_beta)
tprops_gamma = props_TP(ndeb_gamma, V0_gamma)

```

Each list holds, for each pressure, the dictionary of properties; the Gibbs energy is the key 'G'. The stable phase at each (T, P) is the one with the lowest G (1 = α , 2 = β , 3 = γ):

```

G_z = np.zeros((len(Ts), len(Ps)))
for i in range(len(Ts)):
    for j in range(len(Ps)):
        G_list = [tprops_alpha[j]['G'][i], tprops_beta[j]['G'][i], tprops_gamma[j]['G
→'][i]]
        G_z[i, j] = G_list.index(min(G_list)) + 1

```

(this assumes every phase is stable at every (T, P); otherwise compare only the temperatures that min_G returned for all three).

This can be plotted in a P vs T predominance diagram:

4.4. Simultaneous parameter adjusting to experimental heat capacity and thermal expansion at P250 and prediction of thermodynamic phase equilibria at high pressure

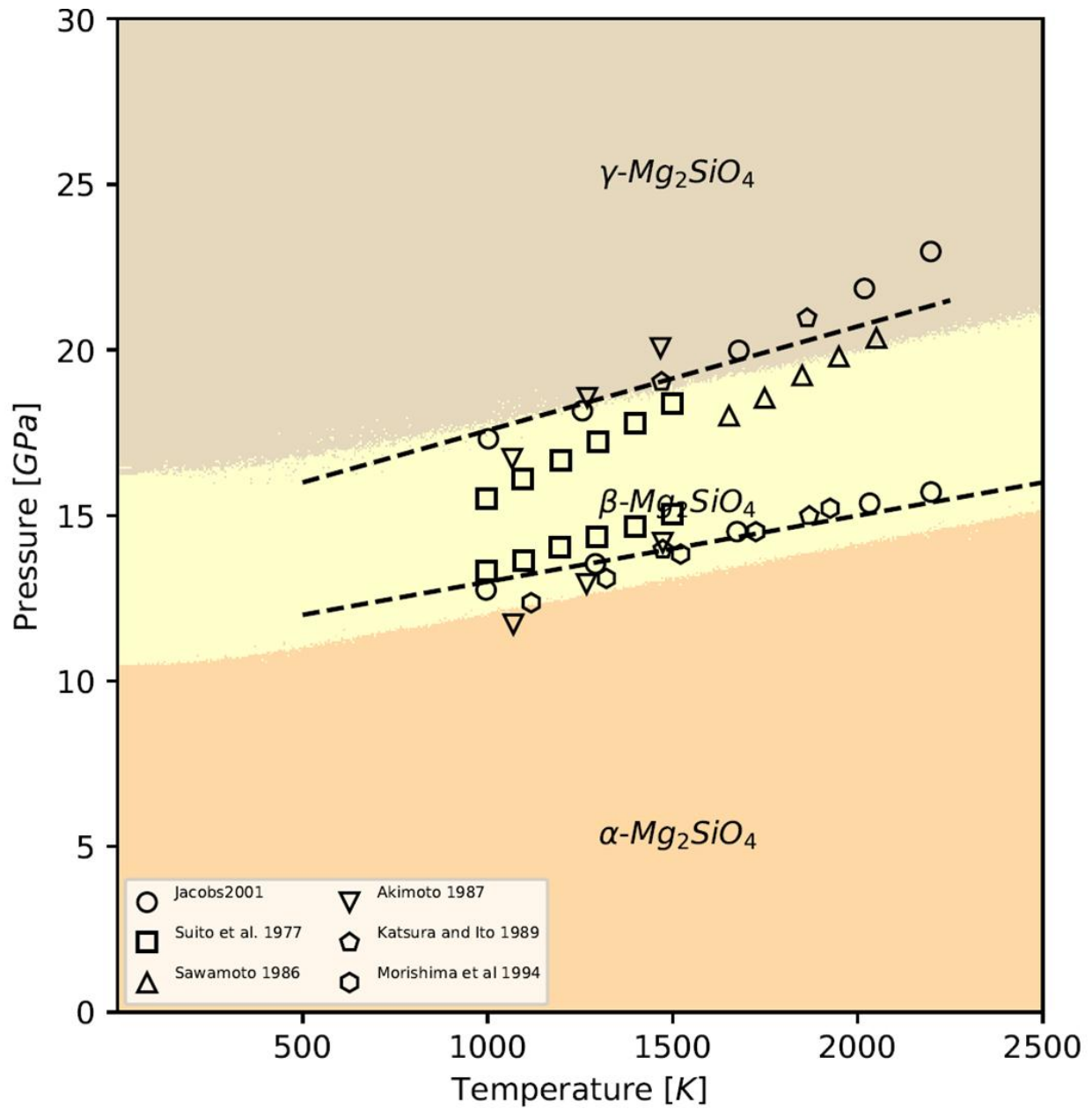


Fig. 5: Phase diagram P versus T for the α , β and γ forms of Mg_2SiO_4 . Symbols are literature data for the phase stability regions boundaries.

THERMODYNAMIC PROPERTIES

Table of contents

- *Units*
- *EOS parametrization*
 - *The EOS's*
 - *Example*
 - *Source code*
- *Poisson's ratio*
 - *Example*
 - *Source Code*
- *Thermodynamic Properties*
 - *Example: minimization of the Gibbs energy*
 - *Example: evaluation of the thermodynamic properties*
 - *Source code*

5.1 Units

All inputs and outputs of the library are SI per mole of atoms: temperature in K, volume in $\text{m}^3/\text{mol-at}$, energies in J/mol-at , pressures and moduli in Pa, mass in kg/mol-at (mean atomic mass). Results per formula unit are obtained by multiplying by the number of atoms in the formula. The VASP loaders convert to these units (`load_V_E(..., units='J/mol')`); `debyetools.constants` holds the constants and the conversion factors used everywhere (e.g. `EV_ATOM_TO_J_MOL`, `A3_ATOM_TO_M3_MOL`).

5.2 EOS parametrization

5.2.1 The EOS's

The EOS implemented are:

- Third order Birch-Murnaghan (BM; BM3 is a deprecated alias)
- Rose-Vinet (RV)
- Mie-Gruneisen (MG)
- Tight-binding second-moment-approximation (TB)
- Murnaghan (MU)
- Poirier-Tarantola (PT)

Two descriptions of the internal energy through inter-atomic potentials have been included as well; they are built from the crystal structure (formula, cell, basis, cut-off radius and number of neighbour shells, see [pair analysis](#)):

- Morse (MP): three parameters (D, a, r0) per pair type
- EAM (EAM): six parameters per pair type and four per element type

The parameters can be entered by the user (`fit=False`) or fitted to energy-volume data. `fitEOS` returns the parameters and also stores them in `pEOS`; `V0` is the equilibrium volume. Each fit is checked (rms below `rel_tol` times the energy range of the data, a minimum with a positive bulk modulus near the data); if it is not acceptable other starting points are tried, up to `max_starts`. When none is acceptable `EOSFitError` is raised (`on_failure='raise'`, default) or the best attempt is kept with a warning (`on_failure='warn'`). The analytic EOS need no initial parameters (the start is taken from the data); `fit_info` records the accepted start and all attempts.

5.2.2 Example

E(V) data in $\text{\AA}^3$ and eV per atom converted to SI per mol-atom:

```
>>> import numpy as np
>>> import debyetools.potentials as potentials
>>> from debyetools.constants import A3_ATOM_TO_M3_MOL, EV_ATOM_TO_J_MOL
>>> V_data = np.array([11.89,12.29,12.70,13.12,13.55,13.98,14.43,14.88,15.35,15.82,16.
↳31,16.80,17.31,17.82,18.34,18.88,19.42,19.98,20.54,21.12,21.71]) * A3_ATOM_TO_M3_MOL
>>> E_data = np.array([-2.97,-3.06,-3.14,-3.20,-3.26,-3.30,-3.33,-3.36,-3.37,-3.38,-3.
↳38,-3.38,-3.37,-3.36,-3.34,-3.32,-3.30,-3.27,-3.24,-3.21,-3.17]) * EV_ATOM_TO_J_MOL
>>> eos = potentials.BM()
>>> p_EOS = eos.fitEOS(V_data, E_data)
>>> print('E0 = %.4e J/mol-at, V0 = %.4e m3/mol-at, K0 = %.3e Pa, K0p = %.2f' %_
↳tuple(p_EOS))
E0 = -3.2641e+05 J/mol-at, V0 = 9.8032e-06 m3/mol-at, K0 = 6.299e+10 Pa, K0p = 4.51
>>> eos.fit_info['accepted']
'from data'
```

5.2.3 Source code

The following is the source for the description of the third order Birch-Murnaghan EOS:

```
class debyetools.potentials.BM (*args, units='J/mol', parameters="")
```

Third order Birch-Murnaghan EOS and derivatives.

```
E0 (V: float | numpy.ndarray) → float | numpy.ndarray
```

Internal energy.

Parameters

V (float | np.ndarray) – Volume.

Returns

E0(V)

Return type

float|np.ndarray

```
E04min (V: float, pEOS: ndarray) → float
```

Energy for minimization.

Parameters

- **V** (float) – Volume.
- **pEOS** (np.ndarray) – parameters.

Returns

E0.

Return type

float

```
d2E0dV2_T (V: float | numpy.ndarray) → float | numpy.ndarray
```

Internal energy second volume derivative.

Parameters

V (float | np.ndarray) – Volume.

Returns

d2E0dV2_T(V)

Return type

float|np.ndarray

```
d3E0dV3_T (V: float | numpy.ndarray) → float | numpy.ndarray
```

Internal energy third volume derivative.

Parameters

V (float | np.ndarray) – Volume.

Returns

d3E0dV3_T(V)

Return type

float|np.ndarray

```
d4E0dV4_T (V: float | numpy.ndarray) → float | numpy.ndarray
```

Internal energy fourth volume derivative.

Parameters

V (float | np.ndarray) – Volume.

Returns`d4E0dV4_T(V)`**Return type**`float|np.ndarray`**d5E0dV5_T** (*V: float | numpy.ndarray*) → `float | numpy.ndarray`

Internal energy fifth volume derivative.

Parameters**V** (*float | np.ndarray*) – Volume.**Returns**`d5E0dV5_T(V)`**Return type**`float|np.ndarray`**d6E0dV6_T** (*V: float | numpy.ndarray*) → `float | numpy.ndarray`

Internal energy sixth volume derivative.

Parameters**V** (*float | np.ndarray*) – Volume.**Returns**`d6E0dV6_T(V)`**Return type**`float|np.ndarray`**dE0dV_T** (*V: float | numpy.ndarray*) → `float | numpy.ndarray`

Internal energy volume derivative.

Parameters**V** (*float | np.ndarray*) – Volume.**Returns**`dE0dV_T(V)`**Return type**`float|np.ndarray`**error2min** (*P: ndarray, Vdata: ndarray, Edata: ndarray*) → `ndarray`

Error for minimization.

Parameters

- **P** (*np.ndarray*) – E0 parameters.
- **Vdata** (*np.ndarray*) – Volume data.
- **Edata** (*np.ndarray*) – Energy data.

Returns

Error.

Return type`np.ndarray`**fitEOS** (*Vdata: ndarray, Edata: ndarray, initial_parameters: ndarray = None, fit: bool = True, rel_tol: float = 0.02, max_starts: int = 12, on_failure: str = 'raise'*) → `ndarray`

Parameters fitting.

Parameters

- **Vdata** (*np.ndarray*) – Input volume data.
- **Edata** (*np.ndarray*) – Target energy data.
- **initial_parameters** (*np.ndarray*) – Initial guess.
- **fit** (*bool.*) – True to run the fitting. False to just use the input parameters.
- **rel_tol** – A fit is accepted when its rms is below *rel_tol* times the energy range of the data (and it has a minimum with $B0 > 0$ within a factor 2 of the data volumes); otherwise other starting points are tried.
- **max_starts** – Maximum number of starting points.
- **on_failure** – ‘raise’ (EOSFitError, default) or ‘warn’ (keep the best attempt) when no start passes.

Returns

Optimal parameters.

Return type

np.ndarray

The other potentials are:

class *debyetools.potentials.RV* (**args, units='J/mol', parameters=""*)
Rose-Vinet EOS and derivatives.

class *debyetools.potentials.TB* (**args, units='J/mol', parameters=""*)
Thight-Binding second-order-approximation EOS and derivatives.

class *debyetools.potentials.MG* (**args, units='J/mol', parameters=""*)
Mie-Gruneisen EOS and derivatives.

class *debyetools.potentials.MU* (**args, units='J/mol', parameters=""*)
Murnaghan EOS and derivatives.

class *debyetools.potentials.PT* (**args, units='J/mol', parameters=""*)
Poirier-Tarantola EOS and derivatives.

class *debyetools.potentials.MP* (**args, units='J/mol', parameters="", prec=10*)
Morse potential and derivatives.

Parameters

- **args** (*list*) – formula, primitive_cell, basis_vectors, cutoff, number_of_neighbor_levels.
- **parameters** (*list_of_floats*) – Morse potential parameters.

EO (*V: float | numpy.ndarray*) → *float | numpy.ndarray*

Internal energy.

Parameters

V (*float | np.ndarray*) – Volume.

Returns

$E0(V)$

Return type

float | np.ndarray

fitEOS (*Vdata*: ndarray, *Edata*: ndarray, *initial_parameters*: ndarray = None, *fit*: bool = True, *rel_tol*: float = 0.02, *max_starts*: int = 12, *on_failure*: str = 'raise') → ndarray

Parameters fitting.

Parameters

- **Vdata** (*np.ndarray*) – Input volume data.
- **Edata** (*np.ndarray*) – Target energy data.
- **initial_parameters** (*np.ndarray*) – Initial guess.
- **fit** (*bool*.) – True to run the fitting. False to just use the input parameters.
- **rel_tol** – A fit is accepted when its rms is below *rel_tol* times the energy range of the data (and it has a minimum with $B_0 > 0$ within a factor 2 of the data volumes); otherwise other starting points are tried.
- **max_starts** – Maximum number of starting points.
- **on_failure** – 'raise' (EOSFitError, default) or 'warn' (keep the best attempt) when no start passes.

Returns

Optimal parameters.

Return type

np.ndarray

class *debyetools.potentials.EAM* (*args, units='J/mol', parameters="")

EAM potential and derivatives.

Parameters

- **args** (*Tuple*[str, *np.ndarray*, *np.ndarray*, float, int]) – formula, primitive_cell, basis_vectors, cutoff, number_of_neighbor_levels.
- **parameters** (*np.ndarray*) – EAM potential parameters.

E0 (*V*)

Internal energy.

Parameters

V (*float*) – Volume.

Return float

Energy.

fitEOS (*Vdata*: ndarray, *Edata*: ndarray, *initial_parameters*: ndarray = None, *fit*: bool = True, *rel_tol*: float = 0.02, *max_starts*: int = 12, *on_failure*: str = 'raise') → ndarray

Parameters fitting.

Parameters

- **Vdata** (*np.ndarray*) – Input volume data.
- **Edata** (*np.ndarray*) – Target energy data.
- **initial_parameters** (*np.ndarray*) – Initial guess.
- **fit** (*bool*.) – True to run the fitting. False to just use the input parameters.
- **rel_tol** – A fit is accepted when its rms is below *rel_tol* times the energy range of the data (and it has a minimum with $B_0 > 0$ within a factor 2 of the data volumes); otherwise other starting points are tried.

- **max_starts** – Maximum number of starting points.
- **on_failure** – ‘raise’ (EOSFitError, default) or ‘warn’ (keep the best attempt) when no start passes.

Returns

Optimal parameters.

Return type

np.ndarray

class debyetools.potentials.EOSFitErrorRaised by fitEOS when no starting point gives an acceptable fit (see `_fit_with_restarts`).

5.3 Poisson’s ratio

The Poisson’s ratio used in the calculation of the Debye temperature can be entered manually by the user or calculated from the elastic moduli matrix (Voigt-Reuss-Hill averages). `load_EM` reads a VASP IBRION = 6 OUTCAR and returns the matrix in kBar, in VASP order (XX, YY, ZZ, XY, YZ, ZX); by default the relaxed-ion moduli (`block='relaxed'`), or the clamped-ion ones with `block='clamped'`. With `quiet=True`, `poisson_ratio` returns all the averages.

5.3.1 Example

```
>>> import os
>>> import debyetools
>>> from debyetools.aux_functions import load_EM
>>> from debyetools.poisson import poisson_ratio
>>> folder = os.path.join(os.path.dirname(debyetools.__file__), 'examples', 'Al_fcc')
>>> EM = load_EM(folder + '/OUTCAR_elastic')
>>> print('%.4f' % poisson_ratio(EM))
0.3370
>>> B_R, B_V, B, G_R, G_V, G, A_U, nu = poisson_ratio(EM, quiet=True)
>>> print('B = %.1f GPa, G = %.1f GPa, A_U = %.2f' % (B, G, A_U))
B = 78.0 GPa, G = 28.5 GPa, A_U = 1.80
```

5.3.2 Source Code

`debyetools.poisson.poisson_ratio` (*EM: ndarray, quiet: bool = False*) → Union[float, Tuple[float, float, float, float, float, float, float, float]]

Poisson’s ratio from the elastic moduli (stiffness) matrix, Voigt-Reuss-Hill average: $\nu = (3K - Y)/(6K)$, $Y = 9KG/(3K + G)$, $K = (K_V + K_R)/2$, $G = (G_V + G_R)/2$.

The Reuss bounds are computed from the compliance $S = C^{-1}$, which is exact for any crystal symmetry (cubic ... triclinic) and independent of the order of the shear components (VASP or Voigt).

Parameters

- **EM** (*np.ndarray*) – 6x6 elastic moduli matrix. `nu` is unit-free; with `quiet=True` the input is taken in kBar (as returned by `aux_functions.load_EM` and `get_elastic.get_EM`) and the moduli are returned in GPa.
- **quiet** (*bool*) – if True, return all VRH quantities (see `quiet_pa`) instead of `nu` only.

Returns

Poisson's ratio, or (B_R, B_V, B, G_R, G_V, G, A_U, nu) if quiet.

Return type

float

`debyetools.poisson.quiet_pa(EM: ndarray) → Tuple[float, float, float, float, float, float, float, float]`

Voigt-Reuss-Hill quantities from the elastic moduli matrix.

Parameters

EM (`np.ndarray`) – 6x6 elastic moduli matrix in kBar (as returned by `load_EM` / `get_EM`).

Returns

B_R, B_V, B, G_R, G_V, G (bulk and shear moduli in GPa), A_U (universal anisotropy index), nu

Return type

Tuple[float, float, float, float, float, float, float, float]

5.4 Thermodynamic Properties

The thermodynamic properties are calculated by first creating an `nDeb` object with the parameters of every contribution, $F = E_0 + F_{\text{vib}} + F_{\text{el}} + F_{\text{def}} + F_{\text{anh}} + F_{\text{xs}}$ (see [contributions](#)). `nDeb.min_G` gives the equilibrium volume at each temperature and pressure; `nDeb.eval_props` evaluates the properties at those (T, V). If no mechanically stable volume exists at some temperature, `min_G` stops there with a warning and returns the stable part; `nDeb.min_G_info` holds the status of every temperature.

5.4.1 Example: minimization of the Gibbs energy

```
>>> from debyetools.aux_functions import load_V_E, gen_Ts
>>> from debyetools.ndeb import nDeb
>>> V, E = load_V_E(folder + '/SUMMARY', folder + '/CONTCAR', units='J/mol')
>>> eos = potentials.BM()
>>> p_EOS = eos.fitEOS(V, E)
>>> p_intanh, p_electronic, p_defects, p_anh = (0, 1), (0, 0, 0, 0), (1e10, 0, 933, 0.
↪1), (0, 0, 0)
>>> ndeb = nDeb(poisson_ratio(EM), 0.0269815385, p_intanh, eos, p_electronic, p_
↪defects, p_anh, mode='jjsl')
>>> T, V = ndeb.min_G(gen_Ts(0.1, 2500, 6), eos.V0, P=0)
>>> print(T)
[1.00000e-01 2.98150e+02 5.00080e+02 1.00006e+03 1.50004e+03]
>>> ndeb.min_G_info['status']
['ok', 'ok', 'ok', 'ok', 'ok', 'unstable', 'not computed']
```

Here the lattice has no stable volume at 2000 K with this EOS, so `min_G` stopped after 1500 K.

5.4.2 Example: evaluation of the thermodynamic properties

`eval_props` returns a dictionary of arrays, for example 'V', 'tD' (Debye temperature, K), 'g' (Grüneisen parameter), 'Kt' and 'Ks' (isothermal and adiabatic bulk moduli, Pa), 'a' (volumetric thermal expansion, 1/K), 'Cv', 'Cp' and 'S' (J/mol-at/K), 'E', 'F', 'G' (J/mol-at), and the individual contributions ('Fvib', 'Fel', 'Fdef', 'Fa', 'Fxs', ...).

```
>>> tprops = ndeb.eval_props(T, V, P=0)
>>> i = list(T).index(298.15)
>>> print('tD = %.1f K, g = %.3f, Kt = %.3e Pa, a = %.3e 1/K, Cp = %.3f, S = %.3f J/
↪mol-at/K'
...      % tuple(tprops[k][i] for k in ('tD', 'g', 'Kt', 'a', 'Cp', 'S')))
tD = 403.4 K, g = 2.184, Kt = 6.983e+10 Pa, a = 7.011e-05 1/K, Cp = 23.842, S = 26.
↪820 J/mol-at/K
```

5.4.3 Source code

```
class debyetools.ndeb.nDeb (nu: float, m: float, p_intanh: ndarray, EOS: object, p_electronic: ndarray,
                           p_defects: ndarray, p_anh: ndarray, *args: object, units: str = 'J/mol', mode:
                           str = 'jjsl', xparams=(0, 0, 0, 0, 0, 0), r=1)
```

Instantiate an object that contains all the parameters for the evaluation of the thermodynamic properties of a certain element or compound. Also contains the method that implements an original Debye formalism for the calculation of the thermodynamic properties.

All quantities are SI per mole of atoms: T in K, V in m³/mol-at, energies in J/mol-at, P and moduli in Pa, mass in kg/mol-at.

$F(T, V) = E_0(V) + F_{\text{vib}} + F_{\text{el}} + F_{\text{def}} + F_{\text{anh}} + F_{\text{xs}}$.

Parameters

- **nu** (*float*) – Poisson's ratio (sets the sound-velocity factor of the Debye temperature).
- **m** (*float*) – Mean atomic mass in kg/mol-at.
- **p_intanh** (*np.ndarray*) – Intrinsic anharmonicity (a_0, m_0): $\theta_D(T, V) = \theta_D(V) \exp(a(V) T / 2)$, $a(V) = a_0 (V/V_0)^{m_0}$, a_0 in 1/K, m_0 dimensionless; V_0 is taken from the EOS (see `intAnharmonicity`).
- **EOS** (*object*) – Equation of state instance (potentials.BM, RV, MG, TB, MU, PT, MP, EAM). Its V_0 and $B_0 = V_0 E_0'(V_0)$ are also used by the vacancy and intrinsic-anharmonicity terms.
- **p_electronic** (*np.ndarray*) – Electronic contribution parameters (q_0, q_1, q_2, q_3), see `electronic.Electronic`.
- **p_defects** (*np.ndarray*) – Mono-vacancy parameters ($E_{\text{vac0}}, S_{\text{vac0}}, T_m, a$): formation energy $E_{\text{vac}} = E_{\text{vac0}} k_B T_m$ and entropy $S_{\text{vac}} = S_{\text{vac0}} k_B$ ($E_{\text{vac0}}, S_{\text{vac0}}$ dimensionless), T_m in K, a dimensionless coefficient of the volume dependence of E_{vac} (see `defects.Defects`).
- **p_anh** (*np.ndarray*) – Explicit anharmonicity (s_0, s_1, s_2): $F_{\text{anh}} = -A(V) T^2 / 2$, $A(V) = s_0 + s_1 V + s_2 V^2$ in J/mol-at/K² (see `anharmonicity.Anharmonicity`).
- **mode** (*str*) – Debye-temperature model: 'jjsl', 'jjdm', 'jjfv', 'SI', 'DM', 'VZ' or 'mfv' (see `vibrational.Vibrational` for definitions).
- **xparams** (*tuple*) – Excess term ($xs_0, \dots, xs_5$): $F_{\text{xs}} = xs_0 + xs_1 T + xs_2 T^2 + xs_3 T^3 + xs_4 T \ln T + xs_5 T^{-2}$ in J/mol-at (see `XS.Xs`). Each coefficient is a number (V-independent).

or a sequence of polynomial coefficients in V (lowest order first, V in $\text{m}^3/\text{mol-at}$) for a V -dependent $A_i(V)$.

- **r** (*float*) – Number of atoms in the chemical formula (Lu et al., Acta Mater. 55 (2007) 1215). Keep the default $r = 1$: with the per-mol-atom inputs above, $r = 1$ is exact (r cancels from θ_D , F_{vib} and F_{def}) and every output is per mol-atom. $r \neq 1$ is only consistent when V , $E0$ and the electronic DOS are given per mole of formula units; then the extensive outputs are per mole of formula units (r times the per-atom values; F_{el} is not scaled by r). A UserWarning is issued when $r \neq 1$.
- **units** (*str*) – Deprecated and ignored. All quantities are SI per mol-atom (J/mol, m^3/mol , kg/mol, Pa).

eval_Cp (T : ndarray, V : ndarray, $P=None$) $\rightarrow$ dict

Constant-pressure heat capacity at (T , V) only (lighter than eval_props: second derivatives only).

Uses the same contributions and factors as eval_props ($E0$, vibrational, electronic, defects, explicit anharmonicity and excess Xs, same assembly as eval_props), so `eval_Cp(T, V)['Cp']` equals `eval_props(T, V)['Cp']`.

Parameters

- **T** (*np.ndarray*) – The temperature in Kelvin.
- **V** (*np.ndarray*) – The volume in $\text{m}^3/\text{mol-at}$.
- **P** – Deprecated and ignored.

Returns

{‘Cp’: constant-pressure heat capacity}.

Return type

dict

eval_props (T : ndarray, V : ndarray, $P=None$) $\rightarrow$ dict

Evaluates the thermodynamic properties of a given compound/element at (T, V).

Parameters

- **T** (*np.ndarray*) – The temperature in Kelvin.
- **V** (*np.ndarray*) – The volume in $\text{m}^3/\text{mol-at}$.
- **P** – Deprecated and ignored; P is computed as $-dF/dV$ at (T, V) and returned under key ‘P’.

Returns

dict of arrays (SI per mol-atom): ‘T’ (K), ‘V’ ($\text{m}^3/\text{mol-at}$), ‘P’ = $-(dF/dV)_T$ (Pa), ‘tD’ Debye temperature (K), ‘g’ Debye-Grueneisen parameter $-\ln(\theta_D)/\ln(V)$ (not the thermodynamic $\gamma = a K_T V / C_V$), ‘Kt’ isothermal bulk modulus (Pa), ‘Ktp’ $(dK_T/dP)_T$, ‘Ktpp’ $(d^2K_T/dP^2)_T$ (1/Pa), ‘Cv’, ‘Cp’ heat capacities (J/mol-at/K), ‘a’ volumetric thermal expansion (1/K), ‘Ks’ adiabatic bulk modulus (Pa), ‘Ksp’ $(dK_S/dP)_T$ at constant T (not $(dK_S/dP)_S$), ‘G’ Gibbs energy $F + PV$, ‘E’ internal energy, ‘E0’ cold energy from the EOS, ‘Fvib’, ‘Evib’ vibrational free / internal energy (J/mol-at), ‘S’, ‘Svib’ entropies and ‘Cvvib’ vibrational heat capacity (J/mol-at/K), ‘Pcold’ = $-dE0/dV$ (Pa), ‘dPdT_V’ (Pa/K), ‘G^2’ = $K_T P^2 - 2 K_T K_{TP}$, ‘dSdP_T’ (J/mol-at/K/Pa), ‘dKdT_P’ (Pa/K), ‘dadP_T’ (1/K/Pa), ‘dCp dP_T’ (J/mol-at/K/Pa), ‘ddSdT_P dP_T’ ($d^2S/dT dP$), plus intermediate quantities (‘dtDdV_T’, ‘d2tDdV2_T’, ‘D_3’, ‘d2E0dV2_T’, ‘dPdV_T’, ‘dE0dV_T’, ‘d3E0dV3_T’, ‘Fa’, ‘Fdef’, ‘Fel’, ‘Sa’, ‘Fxs’).

Return type

dict

f2min (T : float, V : float, P : float) $\rightarrow$ float

free energy for minimization.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.
- **P** (*float*) – Pressure.

Returns

Free energy.

Return type

float

min_G (*T: ndarray, initial_V: float, P: float*) → Tuple[ndarray, ndarray]

Equilibrium volume as a function of temperature at pressure P.

For each temperature the volume is the stable root of $P = -(dF/dV)_T$ (equivalently the minimum of $G = F + PV$) on the branch continued from the previous temperature (from *initial_V* for the first one). The derivatives are the analytic ones used by `eval_props`, so `eval_props(T, V)['P']` equals P to solver precision.

If no mechanically stable solution exists at some temperature ($K_t \leq 0$, θ_D not finite or ≤ 0 , e.g. beyond the EOS spinodal), the calculation stops there: T and V are returned up to the last stable temperature and a UserWarning is issued. Per-temperature details are stored in `self.min_G_info` (keys 'T', 'V', 'P_residual', 'status').

Parameters

- **T** (*list_of_floats*) – Temperature.
- **initial_V** (*float*) – initial guess (if it is not mechanically stable, EOS.V0 is used).
- **P** (*float*) – Pressure.

Returns

Temperature and Volume

Return type

Tuple[np.ndarray, np.ndarray]

INPUT FILES, GRIDS, CONSTANTS

Table of contents

- *Loading VASP results*
- *Temperature and pressure grids*
 - *Source code*
- *Constants and unit conversions*

6.1 Loading VASP results

`load_V_E` (energy-volume curve from a SUMMARY file and the reference CONTCAR), `load_doscar` (total DOS of a series of DOSCAR files), `load_EM` (elastic moduli from an IBRION = 6 OUTCAR) and `load_cell` (formula, cell and fractional basis of a CONTCAR, for the Morse and EAM potentials). File formats: *Input file formats*.

6.2 Temperature and pressure grids

`gen_Ts` (temperatures, 298.15 K added when missing) and `gen_Ps` (pressures).

6.2.1 Source code

`debyetools.aux_functions.gen_Ps(Pi, Pf, nPs)`

Function to generate a range of pressures.

Parameters

- **Pi** (*float*) – Initial pressure.
- **Pf** (*float*) – Final pressure.
- **nPs** (*int*) – Number of values between Pi and Pf, both included.

Returns

Values of pressures from Pi to Pf.

Return type

`np.ndarray`

`debyetools.aux_functions.gen_Ts (Ti: float, Tf: float, nTs: int) → ndarray`

Function to generate a range of temperatures.

Parameters

- **Ti** (*float*) – Initial temperature. (Try not to use the value 0. Use 0.1 instead.)
- **Tf** (*float*) – Final temperature.
- **nTs** (*int*) – Number of values between Ti and Tf, both included. Room temperature (298.15 K) is added if it is not already one of them, so the result has nTs or nTs + 1 values.

Returns

Sorted temperatures.

Return type

np.ndarray

`debyetools.aux_functions.load_EM (filename_outcar_eps: str, block: str = 'relaxed') → ndarray`

Extract the stiffness tensor from the VASP output (OUTCAR for IBRION=6, ISIF>=3).

The matrix is returned in kBar, in VASP order (XX, YY, ZZ, XY, YZ, ZX).

Parameters

- **filename_outcar_eps** (*str*) – file path.
- **block** (*str*) – ‘relaxed’ (default) for the relaxed-ion stiffness, i.e. the VASP block “TOTAL ELASTIC MODULI”; if that block is absent it is computed as “SYMMETRIZED ELASTIC MODULI” + “ELASTIC MODULI CONTR FROM IONIC RELAXATION”. ‘clamped’ for the clamped-ion stiffness (“SYMMETRIZED ELASTIC MODULI”), which was the behaviour before v2.9 (review decision D1).

Returns

Stiffness tensor (kBar).

Return type

np.ndarray

`debyetools.aux_functions.load_V_E (energy_dir_summary: str, energy_dir_contcar: str, units: str = 'eV/atom', energy: str = 'E0') → tuple[numpy.ndarray, numpy.ndarray]`

Loads Energy curve as function of volume from VASP outputs.

The reference volume per atom is the cell volume of the POSCAR/CONTCAR (absolute determinant of the lattice matrix, scale factor applied) divided by the number of atoms. Each SUMMARY line holds the isotropic linear strain d of that calculation relative to the POSCAR/CONTCAR cell in column 1 ($V = V_{\text{ref}} (1 + d)^3$), followed by the last OSZICAR line of the run, e.g. “d 1 F= -.1284E+02 E0= -.1283E+02 d E =... mag=...”. Exact duplicate lines are read once.

The energy is the value after the label ‘E0=’ (energy extrapolated to zero smearing, $\sigma \rightarrow 0$) by default. ‘F=’ is the free energy of the smeared electrons ($E - \sigma S_{\text{el}}$), which contains an electronic entropy term at the artificial electronic temperature σ/kB ; since the electronic free energy is added separately from the DOS (Electronic), E0 is the static lattice energy that belongs in the EOS. `energy=F` reproduces the results of debyetools 2.8.3 and earlier. A line without a usable E0 value (no ‘E0=’ label, or ‘E0= 0’ written as a placeholder) is read from ‘F=’; a line without the requested label is read positionally (column 4, the F value of the standard layout). Both cases give a warning.

Parameters

- **energy_dir_summary** (*str*) – Summary file path.
- **energy_dir_contcar** (*str*) – Atoms positions file path (reference cell, $d = 0$).

- **units** (*str*) – ‘eV/atom’ (V in Å³/atom, E in eV/atom) or ‘J/mol’ (m³/mol-at, J/mol-at).
- **energy** (*str*) – ‘E0’ (default, energy at sigma → 0) or ‘F’ (free energy TOTEN of the smeared electrons).

Returns

Energy as function of volume

Return type

tuple[np.ndarray,np.ndarray]

`debyetools.aux_functions.load_cell` (*filename_contcar*: *str*) → tuple[*str*, numpy.ndarray, numpy.ndarray]

Extract crystal structure from file in VASP format (POSCAR or CONTCAR).

VASP 4 and 5 formats, scale factor, “Selective dynamics” and Direct or Cartesian coordinates are supported (see `_read_poscar`). For VASP 4 files (no element line) the species are named A, B, C, ...

Parameters

filename_contcar (*str*) – File path

Returns

formula (symbols repeated per atom, e.g. ‘AlAlAlLi’), cell (rows = lattice vectors in Å, scale applied) and basis (fractional coordinates).

Return type

tuple[*str*,np.ndarray,np.ndarray]

`debyetools.aux_functions.load_doscar` (*filename_suffix*: *str*, *list_filetags*: *list* = None) → tuple[*list*, *list*, *list*]

Read the total electronic DOS of a series of VASP DOSCAR files (one per volume).

Only the total-DOS block (NEDOS lines after the 6-line header) is read; projected blocks are ignored. For spin-polarised files (ISPIN = 2, 5 columns) the two spin channels are summed; for ISPIN = 1 (3 columns) the DOS column is used. The DOS is divided by the number of atoms.

Parameters

- **filename_suffix** (*str*) – common part of the file names (path + prefix).
- **list_filetags** (*list*) – filename tags appended to filename_suffix (default ‘-0.10’ ... ‘0.10’), in the same order as the volumes.

Returns

E (energy grids, eV), N (total DOS, states/eV/atom), Ef (Fermi levels, eV), one entry per file.

Return type

tuple[*list*,*list*,*list*]

6.3 Constants and unit conversions

`debyetools.constants` (exact SI-2019 values):

```
h = 6.62607015e-34          # Planck constant [J s]
hbar = h / (2 * math.pi)   # reduced Planck constant [J s] = 1.0545718176461565e-34
kB = 1.380649e-23           # Boltzmann constant [J/K]
NAv = 6.02214076e23         # Avogadro constant [1/mol]
eV = 1.602176634e-19        # electron volt [J]
R = kB * NAv                # molar gas constant [J/(mol K)]
```

(continues on next page)

(continued from previous page)

```
# unit conversions
EV_ATOM_TO_J_MOL = eV * NAv          # eV/atom    -> J/mol-atom
A3_ATOM_TO_M3_MOL = 1e-30 * NAv      # A^3/atom   -> m^3/mol-atom
EV_A3_TO_GPA = eV * 1e30 / 1e9       # eV/A^3     -> GPa (160.21766...)
KBAR_TO_GPA = 0.1                    # kBar       -> GPa
```

CONTRIBUTIONS TO THE FREE ENERGY

Table of contents

- *Anharmonicity*
 - *Source code*
- *Defects*
 - *Source code*
- *Electronic Contribution*
 - *Source code*
- *Excess contribution*
 - *Source code*
- *Vibrational*
 - *Source code*

7.1 Anharmonicity

Two anharmonic terms are available. The explicit anharmonicity (`p_anh = (s0, s1, s2)` in `nDeb`) adds $F_{anh} = -\frac{1}{2}A(V)T^2$ with $A(V) = s_0 + s_1V + s_2V^2$. The intrinsic anharmonicity (`p_intanh = (a0, m0)`) makes the Debye temperature depend on temperature, $\theta_D(T, V) = \theta_D(V) \exp(a(V)T/2)$ with $a(V) = a_0(V/V_0)^{m_0}$; (0, 1) switches it off.

7.1.1 Source code

class `debyetools.anharmonicity.Anharmonicity` (`s0: float, s1: float, s2: float`)

Explicit anharmonic contribution to the free energy (J/mol-at): $F_{anh} = -A(V) T^2 / 2$, $A(V) = s_0 + s_1 V + s_2 V^2$.

Parameters

s0, s1, s2 (*float*) – Parameters of $A(V)$: s_0 in J/mol-at/K², s_1 per (m³/mol-at), s_2 per (m³/mol-at)².

A (*V: float*) → *float*

$A(V) = s_0 + s_1V + s_2V^2$, the volume-dependent coefficient of the explicit anharmonic free energy $F_{anh} = -A(V) T^2 / 2$.

Parameters

$\mathbf{V}(\text{float})$ – Volume in $\text{m}^3/\text{mol-at}$.

Returns

$s_0 + s_1 * \mathbf{V} + s_2 * \mathbf{V}^2$

Return type

float

$\mathbf{E}(T, V: \text{float}) \rightarrow \text{float}$

Internal energy due the excess term.

Parameters

- $\mathbf{T}(\text{float})$ – Temperature.
- $\mathbf{V}(\text{float})$ – Volume.

Returns

$A(\mathbf{V}) * \mathbf{T}^2 / 2$

Return type

float

$\mathbf{F}(T, V: \text{float}) \rightarrow \text{float}$

Free energy due the excess term.

Parameters

- $\mathbf{T}(\text{float})$ – Temperature.
- $\mathbf{V}(\text{float})$ – Volume.

Returns

$-A(\mathbf{V}) * \mathbf{T}^2 / 2$

Return type

float

$\mathbf{S}(T, V: \text{float}) \rightarrow \text{float}$

Entropy due the excess term.

Parameters

- $\mathbf{T}(\text{float})$ – Temperature.
- $\mathbf{V}(\text{float})$ – Volume.

Returns

$A(\mathbf{V}) * \mathbf{T}$

Return type

float

$\mathbf{d2AdV2_T}(V: \text{float}) \rightarrow \text{float}$

Second order volume derivative of A at fixed T.

Parameters

$\mathbf{V}(\text{float})$ – Volume

Returns

$2 * s_2$

Return type

float

d2FdT2_V (*T*, *V*: *float*) → *float*

Second order Temperature derivative of the free energy due the excess term, at fixed V.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

Returns

-A(V)

Return type

float

d2FdV2_T (*T*, *V*: *float*) → *float*

Second order volume derivative of the free energy due the excess term, at fixed T.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

Returns

-(d2A(V)/dV2)_T*T**2/2

Return type

float

d2FdVdT (*T*, *V*: *float*) → *float*

Second order derivative of the free energy due the excess term, with respect to T and V.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

Returns

-(dA(V)/dV)_T*T

Return type

float

d3AdV3_T (*V*: *float*) → *float*

Third order volume derivative of A at fixed T.

Parameters

V (*float*) – Volume

Returns

0

Return type

float

d3FdV2dT (*T*, *V*: *float*) → *float*

Third order derivative of the free energy due the excess term, with respect to T, V, and V.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

Returns $-(d^2A(V)/dV^2)_T \cdot T$ **Return type**

float

d3FdV3_T (*T*, *V*: float) → float

Third order volume derivative of the free energy due the excess term, at fixed T.

Parameters

- **T** (float) – Temperature.
- **V** (float) – Volume.

Returns $-(d^3A(V)/dV^3)_T \cdot T^{**2}/2$ **Return type**

float

d3FdVdT2 (*T*, *V*: float) → float

Third order derivative of the free energy due the excess term, with respect to T, T, and V.

Parameters

- **T** (float) – Temperature.
- **V** (float) – Volume.

Returns $-(dA(V)/dV)_T$ **Return type**

float

d4AdV4_T (*V*: float) → float

Fourth order volume derivative of A at fixed T.

Parameters**V** (float) – Volume.**Returns**

0

Return type

float.

d4FdV4_T (*T*, *V*: float) → float

Fourth order volume derivative of the free energy due the excess term, at fixed T.

Parameters

- **T** (float) – Temperature.
- **V** (float) – Volume.

Returns $-(d^4A(V)/dV^4)_T \cdot T^{**2}/2$ **Return type**

float

dAdV_T (*V: float*) → float

Volume derivative of A at fixed T.

Parameters

V (*float*) – Volume

Returns

$s1 + 2 * V * s2$

Return type

float

dFdT_V (*T, V: float*) → float

Temperature derivative of the free energy due the excess term, at fixed V.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

Returns

$-A(V) * T$

Return type

float

dFdV_T (*T, V: float*) → float

Volume derivative of the free energy due the excess term, at fixed T.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

Returns

$-(dA(V)/dV)_T * T^{**2}/2$

Return type

float

class debyetools.anharmonicity.**intAnharmonicity** (*a0: float = 0, m0: float = 1, V0: float = 1*)

Intrinsic anharmonicity: temperature dependence of the Debye temperature, $\theta_D(T, V) = \theta_D(V) * \text{Anh}(T, V)$, $\text{Anh} = \exp(a(V) T / 2)$, $a(V) = a0 (V/V0)^{m0}$.

The derivative methods use values cached by the previous calls (Anh must be called first, then the derivatives in the order used by `vibrational.Vibrational.set_int_anh`).

Anh (*T, V: float*) → float

Intrinsic anharmonicity correction to the Debye temperature.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

Returns

Intrinsic anharmonicity correction

Return type

float

an (*V*: *float*) → *float*

Intrinsic anharmonicity parameter.

Parameters

V (*float*) – Volume

Returns

$a_0 \cdot (V/V_0)^{m_0}$

Return type

float

d2AnhdT2_V () → *float*

Second order temperature derivative of the intrinsic anharmonicity correction of the Debye temperature, at fixed V.

Parameters

V (*float*) – Volume

Returns

$an_val \cdot (dAnh/dT)_V / 2$

Return type

float

d2AnhdV2_T (*T*: *float*, *V*: *float*) → *float*

Second order Volume derivative of the intrinsic anharmonicity correction of the Debye temperature, at fixed T.

Parameters

- **T** (*float*) – Temperature
- **V** (*float*) – Volume

Returns

$dAnhdV_T_val \cdot (T \cdot an_val \cdot m_0 + 2 \cdot m_0 - 2) / (2 \cdot V)$

Return type

float

d2AnhdVdT (*T*: *float*) → *float*

Second order derivative of the intrinsic anharmonicity correction of the Debye temperature, with respect to T and V.

Parameters

T (*float*) – Temperature

Returns

$dAnhdV_T_val \cdot (1 / T + an_val / 2)$

Return type

float

d3AnhdT3_V () → *float*

Third order temperature derivative of the intrinsic anharmonicity correction of the Debye temperature, at fixed V.

Parameters

V (*float*) – Volume

Returns

$an_val \cdot (d2Anh/dT^2)_V / 2$

Return type

float

d3AnhdV2dT (*T*: float, *V*: float) → float

Third order derivative of the intrinsic anharmonicity correction of the Debye temperature, with respect to *T*, *V*, and *V*.

Parameters

- **T** (float) – Temperature
- **V** (float) – Volume

Returns

$$dAnhdV_T_val/V*(m0/T-1/T+3/2*an_val*m0-an_val/2+an_val^2*m0*T/4)$$
Return type

float

d3AnhdV3_T (*T*: float, *V*: float) → float

Third order Volume derivative of the intrinsic anharmonicity correction of the Debye temperature, at fixed *T*.

Parameters

- **T** (float) – Temperature
- **V** (float) – Volume

Returns

$$dAnhdV_T_val*(T^2*an_val^2*m0^2+6*T*an_val*m0^2-6*T*an_val*m0+4*m0^2-12*m0+8)/(4*V^2)$$
Return type

float

d3AnhdVdT2 (*T*: float) → float

Third order derivative of the intrinsic anharmonicity correction of the Debye temperature, with respect to *T*, *T*, and *V*.

Parameters

T (float) – Temperature

Returns

$$dAnhdV_T_val * an_val * (1 / T + an_val / 4)$$
Return type

float

d4AnhdT4_V () → float

Fourth order temperature derivative of the intrinsic anharmonicity correction of the Debye temperature, at fixed *V*.

Parameters

V (float) – Volume.

Returns

$$an_val*(d3Anh_val/dT3)_V/2$$
Return type

float

d4AnhdV4_T (*T*: float, *V*: float) → float

Fourth order Volume derivative of the intrinsic anharmonicity correction of the Debye temperature, at fixed T.

Parameters

- **T** (*float*) – Temperature
- **V** (*float*) – Volume

Returns

$$\frac{dAnhdV_T_val * (T^3 * an_val^3 * m0^3 + 12 * T^2 * an_val^2 * m0^3 - 12 * T^2 * an_val^2 * m0^2 + 28 * T * an_val * m0^3 - 72 * T * an_val * m0^2 + 44 * T * an_val * m0 + 8 * m0^3 - 48 * m0^2 + 88 * m0 - 48)}{(8 * V^3)}$$

Return type

float

dAnhdT_V () → float

Temperature derivative of the intrinsic anharmonicity correction of the Debye temperature, at fixed V.

Parameters

V (*float*) – Volume

Returns

$an_val * Anh_val / 2$

Return type

float

dAnhdV_T (*T*: float, *V*: float) → float

Volume derivative of the intrinsic anharmonicity correction of the Debye temperature, at fixed T.

Parameters

- **T** (*float*) – Temperature
- **V** (*float*) – Volume

Returns

$m0 * T * an_val * Anh_val / (2 * V)$

Return type

float

7.2 Defects

The defects due to mono-vacancies can be taken into account if the parameters are provided (`p_defects = (Evac00, Svac00, Tm, a)`): formation energy $E_{vac} = Evac00 k_B T_m$, entropy $S_{vac} = Svac00 k_B$, melting temperature T_m in K, and a dimensionless coefficient of the volume dependence of E_{vac}). A very large `Evac00` (e.g. `1e10`) switches the term off.

7.2.1 Source code

class `debyetools.defects.Defects` (*Evac00: float, Svac00: float, Tm: float, a: float, P2: float, V0: float, r: float = 1*)

Contribution of thermally activated mono-vacancies to the free energy (J/mol-at): $F_{\text{def}} = -r N_A k_B T \exp((T S_{\text{vac}} - E_{\text{vac}}(V)) / (k_B T))$ (SM eq. S.61), $E_{\text{vac}}(V) = E_{\text{vac00}} k_B T_m - (V_0 / r) a (V - V_0) P_2 / (N_A V)$ (J per vacancy; $V_0 / (r N_A)$ = volume per atom), $S_{\text{vac}} = S_{\text{vac00}} k_B$.

Parameters

- **Evac00** (*float*) – Formation energy of vacancies in units of $k_B T_m$ (dimensionless).
- **Svac00** (*float*) – Formation entropy of vacancies in units of k_B (dimensionless).
- **Tm** (*float*) – Melting temperature in K.
- **a** (*float*) – Dimensionless coefficient of the volume dependence of E_{vac} .
- **P2** (*float*) – Bulk modulus in Pa (nDeb passes $B_0 = V_0 E_0''(V_0)$ of the EOS).
- **V0** (*float*) – Equilibrium volume in $\text{m}^3/\text{mol-at}$ (nDeb passes EOS.V0).
- **r** (*float*) – Number of atoms in the chemical formula (default 1; keep 1 for per-mol-atom inputs); E, S, F and all derivatives carry the factor r.

E (*T: float, V: float*) → float

E of F_{def} (r times the r = 1 value).

Evac (*V: float*) → float

Enthalpy of formation of vacancies.

Parameters

V (*float*) – Volume.

Returns

$E_f(V)$

Return type

float

F (*T: float, V: float*) → float

F of F_{def} (r times the r = 1 value).

S (*T: float, V: float*) → float

S of F_{def} (r times the r = 1 value).

Svac (*V: float*) → float

Entropy of formation of vacancies.

Parameters

V (*float*) – Volume.

Returns

S_{vac0}

Return type

float

d2EvacdV2_T (*V: float*) → float

Volume-derivative of the enthalpy of formation of vacancies.

Parameters

V (*float*) – Volume.

Returns

Volume-derivative of the enthalpy of formation of vacancies.

Return type

float

d2FdT2_V (*T: float, V: float*) → float

(d2F/dT2)_V of F_def (r times the r = 1 value).

d2FdV2_T (*T: float, V: float*) → float

(d2F/dV2)_T of F_def (r times the r = 1 value).

d2FdVdT (*T: float, V: float*) → float

d2F/dVdT of F_def (r times the r = 1 value).

d2SvacdV2_T (*V: float*) → float

Volume-derivative of the entropy of formation of vacancies.

Parameters

V (*float*) – Volume.

Returns

0

d3EvacdV3_T (*V: float*) → float

Volume-derivative of the enthalpy of formation of vacancies.

Parameters

V (*float*) – Volume.

Returns

Volume-derivative of the enthalpy of formation of vacancies.

Return type

float

d3FdV2dT (*T: float, V: float*) → float

d3F/dV2dT of F_def (r times the r = 1 value).

d3FdV3_T (*T: float, V: float*) → float

(d3F/dV3)_T of F_def (r times the r = 1 value).

d3FdVdT2 (*T: float, V: float*) → float

d3F/dVdT2 of F_def (r times the r = 1 value).

d3SvacdV3_T (*V: float*) → float

Volume-derivative of the entropy of formation of vacancies.

Parameters

V (*float*) – Volume.

Returns

0

d4EvacdV4_T (*V: float*) → float

Volume-derivative of the enthalpy of formation of vacancies.

Parameters

V (*float*) – Volume.

Returns

Volume-derivative of the enthalpy of formation of vacancies.

Return type

float

d4FdV4_T (*T: float, V: float*) → float

(d4F/dV4)_T of F_def (r times the r = 1 value).

d4SvacdV4_T (*V: float*) → float

Volume-derivative of the entropy of formation of vacancies.

Parameters**V** (*float*) – Volume.**Returns**

0

dEvacdV_T (*V: float*) → float

Volume-derivative of the enthalpy of formation of vacancies.

Parameters**V** (*float*) – Volume.**Returns**

Volume-derivative of the enthalpy of formation of vacancies.

Return type

float

dFdT_V (*T: float, V: float*) → float

(dF/dT)_V of F_def (r times the r = 1 value).

dFdV_T (*T: float, V: float*) → float

(dF/dV)_T of F_def (r times the r = 1 value).

dSvacdV_T (*V: float*) → float

Volume-derivative of the entropy of formation of vacancies.

Parameters**V** (*float*) – Volume.**Returns**

0

7.3 Electronic Contribution

The electronic contribution uses the Sommerfeld expression $F_{el} = -\frac{\pi^2}{6} N_A k_B^2 T^2 N(E_F)(V)$, with $N(E_F)(V) = q_0 + q_1 V + q_2 V^2 + q_3 V^3$ the total electronic density of states at the Fermi level (both spins, states/eV/atom) and V in m³/mol-atom. The parameters can be entered manually or fitted with `fit_electronic` to DFT densities of states: by default $N(E_F)$ is read from the total DOS at every volume (`load_doscar` reads the total-DOS block of each VASP DOSCAR and sums the spin channels); with `mode='scaling'` a single DOS at V0 is extended with the free-electron law $N(E_F) \propto (V/V_0)^{2/3}$.

7.3.1 Source code

class `debyetools.electronic.Electronic` (*params: ndarray)

Electronic contribution to the free energy (Sommerfeld, low-T expansion):

$$F_{el}(T, V) = -(\pi^2/6) * N_A * k_B^2 * T^2 * N(E_F)(V) / eV \text{ [J/mol-atom]}$$

with $N(E_F)(V) = q_0 + q_1 * V + q_2 * V^2 + q_3 * V^3$ the total electronic density of states at the Fermi level (both spins summed) in states/eV/atom, and V in m³/mol-atom (so q_1, q_2, q_3 are in states/eV/atom per (m³/mol)^k). The parameters are obtained with `fit_electronic()`. $S_{el} = -dF_{el}/dT$ and $E_{el} = F_{el} + T * S_{el} = +(\pi^2/6) * N_A * k_B^2 * T^2 * N(E_F)/eV$.

Note: the supplementary material of the paper writes the derivatives with $\pi^2/3$ (equivalent to using the DOS of one spin channel); here the total DOS and $\pi^2/6$ are used (review decision D4).

Parameters

params (*float*) – q_0, q_1, q_2, q_3 of $N(E_F)(V)$.

E (*T: float | numpy.ndarray, V: float | numpy.ndarray*) → *float | numpy.ndarray*

Electronic energy

Parameters

- **T** (*float | np.ndarray*) – Temperature.
- **V** (*float | np.ndarray*) – Volume.

Returns

E_{el}

Return type

float | np.ndarray

F (*T: float | numpy.ndarray, V: float | numpy.ndarray*) → *float | numpy.ndarray*

Electronic contribution to the free energy.

Parameters

- **T** (*float | np.ndarray*) – Temperature.
- **V** (*float | np.ndarray*) – Volume.

Returns

F_{el}

Return type

float | np.ndarray

NfV (*V: float*) → *float*

$N(E_f)(V)$

Parameters

V (*float*) – Volume.

Returns

$N(E_f)(V)$

Return type

float

S (*T: float | numpy.ndarray, V: float | numpy.ndarray*) → *float | numpy.ndarray*

Electronic entropy.

Parameters

- T (*float* | *np.ndarray*) – Temperature.
- V (*float* | *np.ndarray*) – Volume.

Returns

S_el

Return type

float|np.ndarray

d2FdT2_V (*T: float* | *numpy.ndarray*, *V: float* | *numpy.ndarray*) → float | *numpy.ndarray*

Derivative of the electronic contribution to the free energy.

Parameters

- T (*float* | *np.ndarray*) – Temperature.
- V (*float* | *np.ndarray*) – Volume.

Returns

F_el

Return type

float|np.ndarray

d2FdV2_T (*T: float* | *numpy.ndarray*, *V: float* | *numpy.ndarray*) → float | *numpy.ndarray*

Derivative of the electronic contribution to the free energy.

Parameters

- T (*float* | *np.ndarray*) – Temperature.
- V (*float* | *np.ndarray*) – Volume.

Returns

F_el

Return type

float|np.ndarray

d2FdVdT (*T: float* | *numpy.ndarray*, *V: float* | *numpy.ndarray*) → float | *numpy.ndarray*

Derivative of the electronic contribution to the free energy.

Parameters

- T (*float* | *np.ndarray*) – Temperature.
- V (*float* | *np.ndarray*) – Volume.

Returns

F_el

Return type

float|np.ndarray

d2NfVdV2_T (*V: float*) → floatderivative of $N(E_f)(V)$ **Parameters** V (*float*) – Volume.**Returns**derivative of $N(E_f)(V)$ **Return type**

float

d3FdV2dT (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray

Derivative of the electronic contribution to the free energy.

Parameters

- **T** (float | np.ndarray) – Temperature.
- **V** (float | np.ndarray) – Volume.

Returns

F_{el}

Return type

float|np.ndarray

d3FdV3_T (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray

Derivative of the electronic contribution to the free energy.

Parameters

- **T** (float | np.ndarray) – Temperature.
- **V** (float | np.ndarray) – Volume.

Returns

F_{el}

Return type

float|np.ndarray

d3FdVdT2 (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray

Derivative of the electronic contribution to the free energy.

Parameters

- **T** (float | np.ndarray) – Temperature.
- **V** (float | np.ndarray) – Volume.

Returns

F_{el}

Return type

float|np.ndarray

d3NfVdV3_T (*V*: float) → float

derivative of N(E_f)(V)

Parameters

V (float) – Volume.

Returns

derivative of N(E_f)(V)

Return type

float

d4FdV4_T (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray

Derivative of the electronic contribution to the free energy.

Parameters

- **T** (float | np.ndarray) – Temperature.
- **V** (float | np.ndarray) – Volume.

Returns

F_el

Return type

float|np.ndarray

d4NfVdV4_T (*V: float*) → floatderivative of $N(E_f)(V)$ **Parameters****V** (*float*) – Volume.**Returns**derivative of $N(E_f)(V)$ **Return type**

float

dFdT_V (*T: float | numpy.ndarray, V: float | numpy.ndarray*) → float | numpy.ndarray

Derivative of the electronic contribution to the free energy.

Parameters

- **T** (*float | np.ndarray*) – Temperature.
- **V** (*float | np.ndarray*) – Volume.

Returns

F_el

Return type

float|np.ndarray

dFdV_T (*T: float | numpy.ndarray, V: float | numpy.ndarray*) → float | numpy.ndarray

Derivative of the electronic contribution to the free energy.

Parameters

- **T** (*float | np.ndarray*) – Temperature.
- **V** (*float | np.ndarray*) – Volume.

Returns

F_el

Return type

float|np.ndarray

dNfVdV_T (*V: float*) → floatderivative of $N(E_f)(V)$ **Parameters****V** (*float*) – Volume.**Returns**derivative of $N(E_f)(V)$ **Return type**

float

debyetools.electronic.N_at_Fermi (*E: ndarray, N: ndarray, Ef: float, sigma: float = 0.0*) → float

Density of states at the Fermi level from one total DOS.

Parameters

- **E** (*np.ndarray*) – Energy grid (eV), increasing.
- **N** (*np.ndarray*) – Total DOS on that grid (states/eV/atom).
- **Ef** (*float*) – Fermi level (eV, same reference as E).
- **sigma** (*float*) – Gaussian broadening (eV). 0 (default): linear interpolation of N at Ef. sigma > 0: Gaussian-weighted average of N around Ef (reduces the noise of a coarse DOS grid; the result depends on sigma).

Returns

N(E_F) in states/eV/atom.

Return type

float

`debyetools.electronic.NfV2m(P: ndarray, Vdata: ndarray, NfVdata: ndarray) → ndarray`

Residuals of the N(E_F)(V) polynomial (not used by fit_electronic any more).

Parameters

- **P** (*np.ndarray*) – Parameters q0..q3.
- **Vdata** (*np.ndarray*) – Volume data.
- **NfVdata** (*np.ndarray*) – N(Ef)(V) data.

Returns

residuals.

Return type

np.ndarray

`debyetools.electronic.NfV_poly_fun(V: float, _A: float, _B: float, _C: float, _D: float) → float`

Polynomial model for N(E_F)(V).

Parameters

- **V** (*float*) – Volume (m³/mol-atom).
- **_A** (*float*) – q0.
- **_B** (*float*) – q1.
- **_C** (*float*) – q2.
- **_D** (*float*) – q3.

Returns

N(E_F) (states/eV/atom).

Return type

float

`debyetools.electronic.fit_electronic(Vs: ndarray, p_el: ndarray, E: list, N: list, Ef: list, ixss: int = 0, ixse: int = None, mode: str = 'dos', V0: float = None, sigma: float = 0.0, order: int = 3) → ndarray`

Fit N(E_F)(V) = q0 + q1*V + q2*V² + q3*V³ for *Electronic*.

mode='dos' (default): N(E_F) is read from the total DOS at every volume (each DOS at its own Fermi level, see `N_at_Fermi()`).

mode='scaling': free-electron scaling from one reference DOS, N(E_F)(V) = N(E_F)(V_ref) * (V/V_ref)^(2/3). V0 is required: with a single DOS (len(E) == 1) V_ref = V0; with one DOS per volume, the DOS of the volume closest to V0 is used and V_ref is that volume. Only that DOS is read.

The polynomial is obtained by linear least squares in the reduced volume $x = V/V_m - 1$ (V_m = mean of the fitted volumes) and converted to coefficients in V .

Parameters

- **Vs** (*np.ndarray*) – Volumes ($\text{m}^3/\text{mol-atom}$), in the same order as E, N, Ef.
- **p_el** (*np.ndarray*) – Not used (kept for compatibility; the fit is linear and needs no initial guess).
- **E** (*list*) – Energy grid of each total DOS (eV), e.g. from load_doscar.
- **N** (*list*) – Total DOS of each volume (states/eV/atom), e.g. from load_doscar.
- **Ef** (*list*) – Fermi level of each volume (eV).
- **ixss** (*int*) – First volume index used in the fit (default 0).
- **ixse** (*int*) – End index (exclusive, python slice) of the volumes used (default None: up to the last volume).
- **mode** (*str*) – ‘dos’ or ‘scaling’.
- **v0** (*float*) – Reference volume for mode=‘scaling’ ($\text{m}^3/\text{mol-atom}$).
- **sigma** (*float*) – Gaussian broadening of the DOS at E_F in eV (default 0: linear interpolation).
- **order** (*int*) – Polynomial order, 1 to 3 (default 3).

Returns

[q0, q1, q2, q3].

Return type

np.ndarray

7.4 Excess contribution

An excess term in the form of a FactSage-like polynomial (`xparams` in `nDeb`, default all zero), $F_{xs} = A_0 + A_1 T + A_2 T^2 + A_3 T^3 + A_4 T \ln T + A_5 T^{-2}$ in J/mol-at. Each coefficient is a number (volume-independent, the usual case) or a sequence of polynomial coefficients in V (lowest order first), which makes the term contribute to the pressure, bulk modulus and thermal expansion.

7.4.1 Source code

```
class debyetools.XS.Xs (*params)
```

Excess contribution to the free energy (J/mol-at), a FactSage-like polynomial in T whose coefficients may depend on volume ([DT] eq. (5)):

$$F_{xs}(T, V) = A_0(V) + A_1(V) T + A_2(V) T^2 + A_3(V) T^3 + A_4(V) T \ln T + A_5(V) T^{-2}$$

Each coefficient is given either as a number (V -independent, the usual case) or as a sequence of polynomial coefficients in V , lowest order first: $A_i(V) = c_{i0} + c_{i1} V + c_{i2} V^2 + \dots$ with V in $\text{m}^3/\text{mol-at}$. With V -independent coefficients all volume derivatives are zero and F_{xs} does not contribute to the pressure, bulk modulus or thermal expansion.

Parameters

params – xs0..xs5 (A0..A5). Units: A0 J/mol-at, A1 J/mol-at/K, A2 J/mol-at/K², A3 J/mol-at/K³, A4 J/mol-at/K, A5 J K²/mol-at (per (m³/mol)^k for the order-k coefficient of a V polynomial).

E (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray

Excess internal energy, $E_{xs} = F_{xs} + T S_{xs}$.

Parameters

- **T** (float | np.ndarray) – Temperature.
- **V** (float | np.ndarray) – Volume.

Returns

E_{xs}

Return type

float|np.ndarray

F (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray

Excess contribution to the free energy.

Parameters

- **T** (float | np.ndarray) – Temperature.
- **V** (float | np.ndarray) – Volume.

Returns

F_{xs}

Return type

float|np.ndarray

S (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray

Excess entropy, $S_{xs} = -(dF_{xs}/dT)_V$.

Parameters

- **T** (float | np.ndarray) – Temperature.
- **V** (float | np.ndarray) – Volume.

Returns

S_{xs}

Return type

float|np.ndarray

d2FdT2_V (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray
(d² F_{xs}/dT^2)_V

d2FdV2_T (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray
(d² F_{xs}/dV^2)_T

d2FdVdT (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray
 $d^2F_{xs}/dVdT$

d3FdV2dT (*T*: float | numpy.ndarray, *V*: float | numpy.ndarray) → float | numpy.ndarray
 d^3F_{xs}/dV^2dT

```

d3FdV3_T (T: float | numpy.ndarray, V: float | numpy.ndarray) → float | numpy.ndarray
    (d3F_xs/dV3)_T
d3FdVdT2 (T: float | numpy.ndarray, V: float | numpy.ndarray) → float | numpy.ndarray
    d3F_xs/dVdT2
d4FdV4_T (T: float | numpy.ndarray, V: float | numpy.ndarray) → float | numpy.ndarray
    (d4F_xs/dV4)_T
dFdT_V (T: float | numpy.ndarray, V: float | numpy.ndarray) → float | numpy.ndarray
    (dF_xs/dT)_V
dFdV_T (T: float | numpy.ndarray, V: float | numpy.ndarray) → float | numpy.ndarray
    (dF_xs/dV)_T

```

7.5 Vibrational

The vibrational free energy is calculated with the Debye model. The mean atomic mass (kg/mol-at) and the Poisson's ratio are input parameters; the internal energy is passed as an EOS object (`debyetools.potentials`). The Debye temperature follows from the EOS and the Poisson's ratio; its volume dependence is set by `mode`: 'jjsl' (Slater, default), 'jjdm' (Dugdale-MacDonald), 'jjfv' (free volume), and 'Sl', 'DM', 'VZ', 'mfv' (Grüneisen parameter from the EOS with a fixed reference volume).

7.5.1 Source code

```

class debyetools.vibrational.Vibrational (nu: float, EOS_obj: object, m: float, intanh: ndarray,
                                         mode: str, rin=1)

```

Instantiate the vibrational contribution to the free energy and its derivatives for the calculation of the thermodynamic properties.

$F_{\text{vib}} = 3 r N_A k_B [3/8 \theta_D + T \ln(1 - \exp(-\theta_D/T)) - T D_3(\theta_D/T) / 3]$ (J/mol), with r the number of atoms in the chemical formula (default 1; $r = 1$ for per-mol-atom V and $E0$, see `nDeb`); r also enters θ_D through $(6 \pi^2 r N_A / V)^{1/3}$ and $B2 \sim 1/(m r)$. Every derivative of F_{vib} carries the same factor r . $\theta_D(T, V) = \theta_D(V) * \text{Anh}(T, V)$ (intrinsic anharmonicity, see `anharmonicity.intAnharmonicity`).

Debye-temperature models (*mode*); $\gamma = -\text{dln}(\theta_D)/\text{dln}(V)$ is the Debye-Grüneisen parameter:

- 'jjsl', 'jjdm', 'jjfv': $\theta_D(V)$ from the sound velocity of the pressure-corrected curvature $B2(V) = (-V \text{d}P/\text{d}V - (2 \lambda + 2)/3 P) / (V m r)$ of the cold curve at every volume, with $\lambda = -1$ (Slater), 0 (Dugdale-MacDonald), +1 (free volume / Vashchenko-Zubarev). At $V0$ ($P = 0$): $\gamma = B0'/2 - 1/6, -1/2, -5/6$.
- 'Sl', 'DM', 'VZ', 'mfv': scaling form $\theta_D(V) = \theta_{D,0} (V/E0)^{1/2} / (V0/E0)^{1/2} (V/V0)^{-a}$, with the reference volume $V0 = \text{EOS.V0}(V0_DM)$ and $a = -1/6$ (Slater), $-1/2$ (Dugdale-MacDonald), $-5/6$ (Vashchenko-Zubarev) and $a = -0.95$ ('mfv', mean-free-volume value; a separate model, not equal to VZ). At $V0$ each of 'Sl', 'DM', 'VZ' gives the same θ_D and $\gamma = B0'/2 + a$ as 'jjsl', 'jjdm', 'jjfv'.

Parameters

- **nu** (*float*) – Poisson's ratio.
- **EOS_obj** (*potential_instance*) – Equation of state object.
- **m** (*float*) – Mean atomic mass in kg/mol-at.
- **intanh** (*intAnharmonicity_instance*) – Intrinsic anharmonicity object.

- **mode** (*str*) – Debye-temperature model (see above).
- **rin** (*float*) – *r*, number of atoms in the chemical formula (default 1; keep 1 for per-mol-atom inputs).

F (*T: float, V: float*) → float

F of F_vib (*r* times the *r* = 1 value).

d2FdT2_V (*T: float, V: float*) → float

(d2F/dT2)_V of F_vib (*r* times the *r* = 1 value).

d2FdV2_T (*T: float, V: float*) → float

(d2F/dV2)_T of F_vib (*r* times the *r* = 1 value).

d2FdVdT (*T: float, V: float*) → float

d2F/dVdT of F_vib (*r* times the *r* = 1 value).

d3FdV2dT (*T: float, V: float*) → float

d3F/dV2dT of F_vib (*r* times the *r* = 1 value).

d3FdV3_T (*T: float, V: float*) → float

(d3F/dV3)_T of F_vib (*r* times the *r* = 1 value).

d3FdVdT2 (*T: float, V: float*) → float

d3F/dVdT2 of F_vib (*r* times the *r* = 1 value).

d4FdV4_T (*T: float, V: float*) → float

(d4F/dV4)_T of F_vib (*r* times the *r* = 1 value).

dFdT_V (*T: float, V: float*) → float

(dF/dT)_V of F_vib (*r* times the *r* = 1 value).

dFdV_T (*T: float, V: float*) → float

(dF/dV)_T of F_vib (*r* times the *r* = 1 value).

set_int_anh (*T: float, V: float*) → None

Calculates intrinsic anharmonicity correction to the Debye temperature and its derivatives.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

set_theta (*T: float, V: float*) → None

Calculates the Debye Temperature and its derivatives.

Parameters

- **T** (*float*) – Temperature.
- **V** (*float*) – Volume.

FACTSAGE COMPOUND DATABASE PARAMETRIZATION

The calculated thermodynamic properties for each EOS selected are used to fit the models for heat capacity, thermal expansion, bulk modulus and its pressure derivative. The resulting parameters can be used in FactSage as a compound database.

8.1 Example

`fit_FS` fits, between `T_from` and `T_to`, the heat capacity $C_p = c_0 + c_1T + c_2T^{-2} + c_3T^2 + c_4T^{-1/2} + c_5T^{-3}$ ('Cp'; $c_5 = 0$ unless `cp_T3=True`), the thermal expansion $\alpha = a_0 + a_1T + a_2T^{-1} + a_3T^{-2}$ ('a'), the inverse bulk modulus ('1/Ks', a cubic in T) and its pressure derivative ('Ksp', linear in T). Values per mol-atom; multiply by the number of atoms of the formula for a FactSage compound entry. Example for Al fcc (BM, P = 0), as in the [thermodynamic properties example](#):

```
>>> import os
>>> import debyetools
>>> import debyetools.potentials as potentials
>>> from debyetools.aux_functions import load_V_E, load_EM, gen_Ts
>>> from debyetools.poisson import poisson_ratio
>>> from debyetools.ndeb import nDeb
>>> folder = os.path.join(os.path.dirname(debyetools.__file__), 'examples', 'Al_fcc')
>>> V, E = load_V_E(folder + '/SUMMARY', folder + '/CONTCAR', units='J/mol')
>>> eos = potentials.BM()
>>> p_EOS = eos.fitEOS(V, E)
>>> nu = poisson_ratio(load_EM(folder + '/OUTCAR_elastic'))
>>> ndeb = nDeb(nu, 0.0269815385, (0, 1), eos, (0, 0, 0, 0), (1e10, 0, 933, 0.1), (0, 0, 0, 0), mode='jjsl')
>>> T, V = ndeb.min_G(gen_Ts(0.1, 1000, 11), eos.V0, P=0)
>>> tprops = ndeb.eval_props(T, V, P=0)
>>> from debyetools.fs_compound_db import fit_FS
>>> FS_db_params = fit_FS(tprops, 298.15, 1000)
>>> for key, values in FS_db_params.items():
...     print(key, ', '.join('%3e' % v for v in values))
Cp 4.989e+01, -2.029e-02, 9.287e+04, 1.395e-05, -3.847e+02, 0.000e+00
a -2.534e-05, 1.056e-07, 3.807e-02, -5.672e+00
1/Ks 1.295e-11, 2.419e-15, -8.245e-20, 1.046e-21
Ksp 4.760e+00, 9.759e-04
```

8.2 Source code

`debyetools.fs_compound_db.fit_FS` (*tprops: dict, T_from: float, T_to: float, cp_T3: bool = False*) → dict

Procedure for the fitting of FS compound database parameters to thermodynamic properties.

Fits, on the temperatures $T_{\text{from}} \leq T \leq T_{\text{to}}$ (0.005 K tolerance, so $T_{\text{from}}/T_{\text{to}}$ need not be grid points): $C_p = P_0 + P_1 T + P_2 T^{-2} + P_3 T^2 + P_4 T^{-0.5} + P_5 T^{-3}$, $\alpha = Q_0 + Q_1 T + Q_2 T^{-1} + Q_3 T^{-2}$, $1/K_s = R_0 + R_1 T + R_2 T^2 + R_3 T^3$, $K_s' = S_0 + S_1 (T - 298.15) \ln(T/298.15)$. All four models are linear in their coefficients and are solved by linear least squares on column-scaled design matrices (exact minimum).

The T^{-3} coefficient P_5 is poorly determined on typical windows (e.g. 298-1000 K): fitting it gives large, mutually cancelling coefficients. By default (`cp_T3=False`) P_5 is fixed at 0 and the other five are fitted; with `cp_T3=True` all six are fitted. (The former `curve_fit` version left P_5 at its initial value 1.0, i.e. effectively the 5-term fit.)

Parameters

- **tprops** (*dict*) – Dictionary with the thermodynamic properties ('T', 'Cp', 'a', 'Ks', 'Ksp').
- **T_from** (*float*) – Initial temperature.
- **T_to** (*float*) – Final temperature.
- **cp_T3** (*bool*) – fit the T^{-3} term of C_p (default False: $P_5 = 0$).

Returns

Dictionary with the optimal parameters ('Cp' (6 values), 'a', '1/Ks', 'Ksp').

Return type

dict

PAIR ANALYSIS CALCULATION

A pair analysis calculator was implemented where the atom position can be read from a POSCAR file or entered manually. The neighbor list and binning are calculated to build up the pairs list.

9.1 Example

```
>>> import numpy as np
>>> from debyetools.pairanalysis import pair_analysis
>>> formula = 'AaAaBbAa'
>>> cutoff = 10
>>> basis_vectors = np.array([[0,0,0], [.5,.5,0], [.5,0,.5], [0,.5,.5]])
>>> primitive_cell = np.array([[4,0,0], [0,4,0], [0,0,4]])
>>> distances, num_pairs_per_formula, combs_types = pair_analysis(formula, cutoff,
↳basis_vectors, primitive_cell)
>>> print('distances | # of pairs per type')
distances | # of pairs per type
>>> print('          | ' + ' '.join(combs_types))
          | Aa-Aa Aa-Bb Bb-Bb
>>> for d, n in zip(distances, num_pairs_per_formula):
...     print(' %.6f |' % d + ' '.join(' %.2f' % ni for ni in n))
2.828427 | 6.00 6.00 0.00
4.000000 | 4.50 0.00 1.50
4.898979 | 12.00 12.00 0.00
5.656854 | 9.00 0.00 3.00
6.324555 | 12.00 12.00 0.00
6.928203 | 6.00 0.00 2.00
7.483315 | 24.00 24.00 0.00
8.000000 | 4.50 0.00 1.50
8.485281 | 18.00 18.00 0.00
8.944272 | 18.00 0.00 6.00
9.380832 | 12.00 12.00 0.00
9.797959 | 18.00 0.00 6.00
```

9.2 Source code

`debyetools.pairanalysis.neighbor_list` (*size: ndarray, basis: ndarray, cell: ndarray, cutoff: float*) →
Tuple[ndarray, ndarray, ndarray, ndarray, ndarray]

calculate a list *i, j, dij* where *i* and *j* are a pair of atoms of indexes *i* and *j*, respectively, and *dij* is the distance between them.

All image cells that can hold a neighbour within the cut-off are included: for an atom pair in image cell *n*, the fractional component *i* of the pair vector is $n_i + (f_j - f_i)$, and $\text{abs}(r \cdot b_i) \leq \text{cutoff} * \text{norm}(b_i)$ (b_i reciprocal vectors, $a_i \cdot b_j = \delta_{ij}$, $1/\text{norm}(b_i)$ = interplanar spacing). Hence $\text{abs}(n_i) \leq \text{cutoff} * \text{norm}(b_i) + \text{span}_i$, with span_i the spread of the basis in fractional coordinate *i*. This holds for any cell shape and any cut-off.

Parameters

- **size** (*np.ndarray*) – Number of times we are replicating the primitive cell
- **basis** (*np.ndarray*) – atoms position within a single primitive cell (fractional)
- **cell** (*np.ndarray*) – the primitive cell (rows = lattice vectors)
- **cutoff** (*float*) – cut-off distance

Returns

distances, *I, J*, image-cell coordinates, image-cell index of each pair

Return type

Tuple[*np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray*]

`debyetools.pairanalysis.pair_analysis` (*atom_types, cutoff, basis, cell, prec=10, full=False*)

run a pair analysis of a crystal structure of almost any type of symmetry.

Parameters

- **atom_types** (*str*) – the types of each atom in the primitive cell in the same order as the basis vectors.
- **cutoff** (*float*) – cut-off distance
- **basis** (*np.ndarray*) – atoms position within a single primitive cell
- **cell** (*np.ndarray*) – the primitive cell
- **prec** (*int*) – precision.
- **full** (*boolean*) – if True, returns also data of ghost cells.

Returns

pair distance, pair number, pair types

Return type

Tuple[*np.ndarray, np.ndarray, np.ndarray*]

INPUT FILE FORMATS

Table of contents

- *Files and names*
- *Reference structure (CONTCAR)*
- *Energy-volume series (SUMMARY)*
- *Electronic DOS (DOSCAR)*
- *Elastic moduli (OUTCAR)*
- *Direct inputs*

10.1 Files and names

debyetools reads the following VASP results. The file names are free: every loader takes the path, so the names below are only the conventions of the example data.

Content	Loader	tests/ inpt_files	debyetools/ examples
Reference structure	load_V_E, load_cell	CONTCAR.5	CONTCAR
Energy-volume series	load_V_E	SUMMARY.fcc	SUMMARY
Electronic DOS per volume	load_doscar	DOSCAR.EvV. <tag>	DOSCAR.EvV.0121
Elastic moduli	load_EM	OUTCAR.eps	OUTCAR_elastic

A convenient layout is one folder per compound named by the formula and the space group, e.g. Al₂O₃_R3c:

```
Al_fcc/
  CONTCAR.5
  SUMMARY.fcc
  DOSCAR.EvV.-0.10 ... DOSCAR.EvV.-0.01, DOSCAR.EvV.-0.00, DOSCAR.EvV.0.01 ... DOSCAR.EvV.
  ↪ 0.10
  OUTCAR.eps
```

load_V_E returns the volumes and energies per atom: in Å³ and eV with the default units='eV/atom', in m³/mol-at and J/mol-at with units='J/mol' (the units nDeb needs).

10.2 Reference structure (CONTCAR)

The relaxed structure (VASP 5 format with the element line). Its cell volume divided by the number of atoms is the reference volume of the energy-volume series (strain $d = 0$); for the Morse and EAM potentials `load_cell` returns the formula, the cell and the fractional basis.

```
Al4
  1.0000000000000000
  4.0396918604376202    0.0000000000000000    0.0000000000000000
  0.0000000000000000    4.0396918604376202    0.0000000000000000
  0.0000000000000000    0.0000000000000000    4.0396918604376202
Al
  4
Direct
  0.0000000000000000    0.0000000000000000    0.0000000000000000
  0.0000000000000000    0.5000000000000000    0.5000000000000000
  0.5000000000000000    0.0000000000000000    0.5000000000000000
  0.5000000000000000    0.5000000000000000    0.0000000000000000
```

10.3 Energy-volume series (SUMMARY)

One line per fixed-volume calculation of the same cell: column 1 is the isotropic linear strain d relative to the reference cell ($V = V_{\text{ref}} (1 + d)^3$), followed by the last OSZICAR line of that run. The energy of the cell (eV) is the value after $E0=$, the energy extrapolated to zero smearing ($\sigma \rightarrow 0$). The value after $F=$ (VASP's free energy TOTEN, $E - \sigma S$ of the smeared electrons) contains an electronic entropy at the artificial temperature σ/k_B ; `nDeb` adds the electronic free energy separately from the DOS, so it is not used by default. `load_V_E(..., energy='F')` reads it and reproduces the results of `debyetools 2.8.3` and earlier. A line without a usable $E0=$ value (no label, or $E0=$ 0 written as a placeholder) is read from $F=$, and a line without the requested label is read positionally (column 4); both give a warning. Duplicate lines are read once.

```
-0.10 1 F= -.12910797E+02 E0= -.12906918E+02 d E =-.775954E-02 mag= -0.0007
-0.09 1 F= -.13370262E+02 E0= -.13366237E+02 d E =-.804932E-02 mag= -0.0011
-0.08 1 F= -.13758391E+02 E0= -.13754102E+02 d E =-.857638E-02 mag= -0.0010
-0.07 1 F= -.14083100E+02 E0= -.14078383E+02 d E =-.943565E-02 mag= -0.0003
-0.06 1 F= -.14349565E+02 E0= -.14344316E+02 d E =-.104987E-01 mag= 0.0009
-0.05 1 F= -.14563748E+02 E0= -.14558028E+02 d E =-.114391E-01 mag= 0.0019
-0.04 1 F= -.14729909E+02 E0= -.14723867E+02 d E =-.120830E-01 mag= 0.0026
-0.03 1 F= -.14853318E+02 E0= -.14847102E+02 d E =-.124328E-01 mag= 0.0029
-0.02 1 F= -.14936018E+02 E0= -.14929721E+02 d E =-.125950E-01 mag= 0.0028
-0.01 1 F= -.14983118E+02 E0= -.14976823E+02 d E =-.125893E-01 mag= 0.0022
-0.00 1 F= -.14997956E+02 E0= -.14991733E+02 d E =-.124452E-01 mag= 0.0010
 0.01 1 F= -.14984456E+02 E0= -.14978268E+02 d E =-.123746E-01 mag= -0.0002
 0.02 1 F= -.14945044E+02 E0= -.14938772E+02 d E =-.125429E-01 mag= -0.0007
 0.03 1 F= -.14882846E+02 E0= -.14876412E+02 d E =-.128695E-01 mag= -0.0007
 0.04 1 F= -.14799945E+02 E0= -.14793346E+02 d E =-.131984E-01 mag= -0.0006
 0.05 1 F= -.14698295E+02 E0= -.14691567E+02 d E =-.134551E-01 mag= -0.0005
 0.06 1 F= -.14581670E+02 E0= -.14574842E+02 d E =-.136561E-01 mag= -0.0004
 0.07 1 F= -.14451232E+02 E0= -.14444380E+02 d E =-.137049E-01 mag= -0.0006
 0.08 1 F= -.14310295E+02 E0= -.14303519E+02 d E =-.135523E-01 mag= -0.0012
 0.09 1 F= -.14158570E+02 E0= -.14151879E+02 d E =-.133818E-01 mag= -0.0015
 0.10 1 F= -.13997863E+02 E0= -.13991236E+02 d E =-.132555E-01 mag= -0.0016
```

10.4 Electronic DOS (DOSCAR)

One DOSCAR per volume of the series, in the same order as the volumes (`list_filetags` gives the file-name tags in that order). Line 2 starts with the volume per atom in $\text{\AA}^3$ (the GUI uses it to pair each file with its volume); line 6 holds $E_{\max}$, $E_{\min}$, the number of points NEDOS and the Fermi energy. Only the total-DOS block (NEDOS lines after line 6) is read: for spin-polarised calculations (ISPIN = 2: energy, DOS up, DOS down, integrated up, integrated down) the two spins are summed, for ISPIN = 1 the DOS column is used; the DOS is divided by the number of atoms (line 1). Projected blocks, if present, are ignored.

```

 4      4      1      0
0.1599154E+02  0.3999295E-09  0.3999295E-09  0.3999295E-09  0.5000000E-15
1.000000000000000E-004
CAR
unknown system
 20.23257380   -4.13078501   301          8.31659488      1.00000000
 -4.131  0.0000E+00  0.0000E+00  0.0000E+00  0.0000E+00
 -4.050  0.0000E+00  0.0000E+00  0.0000E+00  0.0000E+00
 .
 .
 .
-3.481  0.5925E-06  0.5691E-06  0.4812E-07  0.4622E-07
 .
 .
 .
19.339  0.3717E-02  0.3257E-02  0.1200E+02  0.1200E+02

```

10.5 Elastic moduli (OUTCAR)

The OUTCAR of an IBRION = 6 calculation with ISIF >= 3. `load_EM` returns the 6x6 matrix in kBar, in VASP order (XX, YY, ZZ, XY, YZ, ZX): by default the relaxed-ion moduli (block TOTAL ELASTIC MODULI, or SYMMETRIZED ELASTIC MODULI + ELASTIC MODULI CONTR FROM IONIC RELAXATION when it is absent), or the clamped-ion moduli (SYMMETRIZED ELASTIC MODULI) with `block='clamped'`. The Poisson's ratio and the Voigt-Reuss-Hill averages do not depend on the order.

ELASTIC MODULI CONTR FROM IONIC RELAXATION (kBar)						
Direction	XX	YY	ZZ	XY	YZ	ZX
XX	0.0000	0.0000	0.0000	0.0000	0.0000	-0.0000
YY	0.0000	0.0000	0.0000	-0.0000	0.0000	-0.0000
ZZ	0.0000	0.0000	-0.0000	0.0000	0.0000	-0.0000
XY	0.0000	-0.0000	0.0000	0.0000	-0.0000	-0.0000
YZ	0.0000	0.0000	0.0000	-0.0000	0.0000	-0.0000
ZX	-0.0000	-0.0000	-0.0000	-0.0000	-0.0000	0.0000
TOTAL ELASTIC MODULI (kBar)						
Direction	XX	YY	ZZ	XY	YZ	ZX
XX	4520	1500	1070	200.0000	-0.0000	0.0000
YY	1500	4520	1070	-200.0000	-0.0000	0.0000
ZZ	1070	1070	4540	-0.0000	0.0000	0.0000
XY	200	-200	-0	1320	-0.0000	-0.0000
YZ	-0	-0	0	-0	1320	200

(continues on next page)

(continued from previous page)

ZX	0	0	0	-0	200	1510

10.6 Direct inputs

Instead of files, the data can be entered directly as numpy arrays (SI per mol-atom) or, in the GUI, as plain text.

In the EOS-fit dialog of the GUI the energy curve is entered in $\text{\AA}^3$ and eV per atom (option “eV and A³ (per at)”, default) or in m^3 and J per mol-atom (option “J and m³ (per mol-at)”); for example Al_3Li L1₂:

```
#V  E
1.188839e+01      -2.971644e+00
1.228909e+01      -3.061733e+00
1.269870e+01      -3.138113e+00
1.311730e+01      -3.202274e+00
1.354501e+01      -3.255126e+00
1.398191e+01      -3.297945e+00
1.442811e+01      -3.331133e+00
1.488370e+01      -3.355852e+00
1.534878e+01      -3.372588e+00
1.582345e+01      -3.382024e+00
1.630781e+01      -3.384997e+00
1.680195e+01      -3.382070e+00
1.730598e+01      -3.373913e+00
1.781999e+01      -3.360960e+00
1.834407e+01      -3.343770e+00
1.887833e+01      -3.322563e+00
1.942286e+01      -3.298003e+00
1.997777e+01      -3.270464e+00
2.054315e+01      -3.240472e+00
2.111909e+01      -3.208295e+00
2.170570e+01      -3.174068e+00
```

The elastic constants are entered in GPa in Voigt order (XX, YY, ZZ, YZ, ZX, XY), as pasted by the GUI when an OUTCAR is loaded (which converts from the VASP order); for Al_3Li L1₂:

```
# GPa, Voigt order: XX YY ZZ YZ ZX XY
122.26  35.00  35.00 -0.00  0.00  0.00
35.00  122.22  34.95 -0.00  0.00 -0.00
35.00  34.95  122.22  0.00  0.00  0.00
-0.00  -0.00  0.00  41.47  0.00  0.00
0.00  0.00  0.00  0.00  41.39  0.00
0.00  -0.00  0.00  0.00  0.00  41.47
```

PYTHON MODULE INDEX

d

`debyetools.anharmonicity`, 43
`debyetools.aux_functions`, 39
`debyetools.defects`, 51
`debyetools.electronic`, 54
`debyetools.fs_compound_db`, 64
`debyetools.ndeb`, 35
`debyetools.pairanalysis`, 66
`debyetools.poisson`, 33
`debyetools.vibrational`, 61
`debyetools.XS`, 59

A

`A()` (*debyetools.anharmonicity.Anharmonicity method*), 43
`an()` (*debyetools.anharmonicity.intAnharmonicity method*), 47
`Anh()` (*debyetools.anharmonicity.intAnharmonicity method*), 47
`Anharmonicity` (class in *debyetools.anharmonicity*), 43

B

`BM` (class in *debyetools.potentials*), 29

D

`d2AdV2_T()` (*debyetools.anharmonicity.Anharmonicity method*), 44
`d2AnhdT2_V()` (*debyetools.anharmonicity.intAnharmonicity method*), 48
`d2AnhdV2_T()` (*debyetools.anharmonicity.intAnharmonicity method*), 48
`d2AnhdVdT()` (*debyetools.anharmonicity.intAnharmonicity method*), 48
`d2E0dV2_T()` (*debyetools.potentials.BM method*), 29
`d2EvacdV2_T()` (*debyetools.defects.Defects method*), 51
`d2FdT2_V()` (*debyetools.anharmonicity.Anharmonicity method*), 44
`d2FdT2_V()` (*debyetools.defects.Defects method*), 52
`d2FdT2_V()` (*debyetools.electronic.Electronic method*), 55
`d2FdT2_V()` (*debyetools.vibrational.Vibrational method*), 62
`d2FdT2_V()` (*debyetools.XS.Xs method*), 60
`d2FdV2_T()` (*debyetools.anharmonicity.Anharmonicity method*), 45
`d2FdV2_T()` (*debyetools.defects.Defects method*), 52
`d2FdV2_T()` (*debyetools.electronic.Electronic method*), 55
`d2FdV2_T()` (*debyetools.vibrational.Vibrational method*), 62
`d2FdV2_T()` (*debyetools.XS.Xs method*), 60
`d2FdVdT()` (*debyetools.anharmonicity.Anharmonicity method*), 45
`d2FdVdT()` (*debyetools.defects.Defects method*), 52

`d2FdVdT()` (*debyetools.electronic.Electronic method*), 55
`d2FdVdT()` (*debyetools.vibrational.Vibrational method*), 62
`d2FdVdT()` (*debyetools.XS.Xs method*), 60
`d2NfVdV2_T()` (*debyetools.electronic.Electronic method*), 55
`d2SvacdV2_T()` (*debyetools.defects.Defects method*), 52
`d3AdV3_T()` (*debyetools.anharmonicity.Anharmonicity method*), 45
`d3AnhdT3_V()` (*debyetools.anharmonicity.intAnharmonicity method*), 48
`d3AnhdV2dT()` (*debyetools.anharmonicity.intAnharmonicity method*), 49
`d3AnhdV3_T()` (*debyetools.anharmonicity.intAnharmonicity method*), 49
`d3AnhdVdT2()` (*debyetools.anharmonicity.intAnharmonicity method*), 49
`d3E0dV3_T()` (*debyetools.potentials.BM method*), 29
`d3EvacdV3_T()` (*debyetools.defects.Defects method*), 52
`d3FdV2dT()` (*debyetools.anharmonicity.Anharmonicity method*), 45
`d3FdV2dT()` (*debyetools.defects.Defects method*), 52
`d3FdV2dT()` (*debyetools.electronic.Electronic method*), 56
`d3FdV2dT()` (*debyetools.vibrational.Vibrational method*), 62
`d3FdV2dT()` (*debyetools.XS.Xs method*), 60
`d3FdV3_T()` (*debyetools.anharmonicity.Anharmonicity method*), 46
`d3FdV3_T()` (*debyetools.defects.Defects method*), 52
`d3FdV3_T()` (*debyetools.electronic.Electronic method*), 56
`d3FdV3_T()` (*debyetools.vibrational.Vibrational method*), 62
`d3FdV3_T()` (*debyetools.XS.Xs method*), 60
`d3FdVdT2()` (*debyetools.anharmonicity.Anharmonicity method*), 46
`d3FdVdT2()` (*debyetools.defects.Defects method*), 52
`d3FdVdT2()` (*debyetools.electronic.Electronic method*), 56
`d3FdVdT2()` (*debyetools.vibrational.Vibrational method*), 62

- d3FdVdT2 () (*debyetools.XS.Xs method*), 61
 d3NfVdV3_T () (*debyetools.electronic.Electronic method*), 56
 d3SvacdV3_T () (*debyetools.defects.Defects method*), 52
 d4AdV4_T () (*debyetools.anharmonicity.Anharmonicity method*), 46
 d4AnhdT4_V () (*debyetools.anharmonicity.intAnharmonicity method*), 49
 d4AnhdV4_T () (*debyetools.anharmonicity.intAnharmonicity method*), 49
 d4E0dV4_T () (*debyetools.potentials.BM method*), 29
 d4EvacdV4_T () (*debyetools.defects.Defects method*), 52
 d4FdV4_T () (*debyetools.anharmonicity.Anharmonicity method*), 46
 d4FdV4_T () (*debyetools.defects.Defects method*), 53
 d4FdV4_T () (*debyetools.electronic.Electronic method*), 56
 d4FdV4_T () (*debyetools.vibrational.Vibrational method*), 62
 d4FdV4_T () (*debyetools.XS.Xs method*), 61
 d4NfVdV4_T () (*debyetools.electronic.Electronic method*), 57
 d4SvacdV4_T () (*debyetools.defects.Defects method*), 53
 d5E0dV5_T () (*debyetools.potentials.BM method*), 30
 d6E0dV6_T () (*debyetools.potentials.BM method*), 30
 dAdV_T () (*debyetools.anharmonicity.Anharmonicity method*), 46
 dAnhdT_V () (*debyetools.anharmonicity.intAnharmonicity method*), 50
 dAnhdV_T () (*debyetools.anharmonicity.intAnharmonicity method*), 50
 dE0dV_T () (*debyetools.potentials.BM method*), 30
 debyetools.anharmonicity module, 43
 debyetools.aux_functions module, 39
 debyetools.defects module, 51
 debyetools.electronic module, 54
 debyetools.fs_compound_db module, 64
 debyetools.ndeb module, 35
 debyetools.pairanalysis module, 66
 debyetools.poisson module, 33
 debyetools.vibrational module, 61
 debyetools.XS module, 59
 Defects (class in *debyetools.defects*), 51
 dEvacdV_T () (*debyetools.defects.Defects method*), 53
 dFdT_V () (*debyetools.anharmonicity.Anharmonicity method*), 47
 dFdT_V () (*debyetools.defects.Defects method*), 53
 dFdT_V () (*debyetools.electronic.Electronic method*), 57
 dFdT_V () (*debyetools.vibrational.Vibrational method*), 62
 dFdT_V () (*debyetools.XS.Xs method*), 61
 dFdV_T () (*debyetools.anharmonicity.Anharmonicity method*), 47
 dFdV_T () (*debyetools.defects.Defects method*), 53
 dFdV_T () (*debyetools.electronic.Electronic method*), 57
 dFdV_T () (*debyetools.vibrational.Vibrational method*), 62
 dFdV_T () (*debyetools.XS.Xs method*), 61
 dNfVdV_T () (*debyetools.electronic.Electronic method*), 57
 dSvacdV_T () (*debyetools.defects.Defects method*), 53
- ## E
- E () (*debyetools.anharmonicity.Anharmonicity method*), 44
 E () (*debyetools.defects.Defects method*), 51
 E () (*debyetools.electronic.Electronic method*), 54
 E () (*debyetools.XS.Xs method*), 60
 E0 () (*debyetools.potentials.BM method*), 29
 E0 () (*debyetools.potentials.EAM method*), 32
 E0 () (*debyetools.potentials.MP method*), 31
 E04min () (*debyetools.potentials.BM method*), 29
 EAM (class in *debyetools.potentials*), 32
 Electronic (class in *debyetools.electronic*), 54
 EOSFitError (class in *debyetools.potentials*), 33
 error2min () (*debyetools.potentials.BM method*), 30
 Evac () (*debyetools.defects.Defects method*), 51
 eval_Cp () (*debyetools.ndeb.nDeb method*), 36
 eval_props () (*debyetools.ndeb.nDeb method*), 36
- ## F
- F () (*debyetools.anharmonicity.Anharmonicity method*), 44
 F () (*debyetools.defects.Defects method*), 51
 F () (*debyetools.electronic.Electronic method*), 54
 F () (*debyetools.vibrational.Vibrational method*), 62
 F () (*debyetools.XS.Xs method*), 60
 f2min () (*debyetools.ndeb.nDeb method*), 36
 fit_electronic () (in module *debyetools.electronic*), 58
 fit_FS () (in module *debyetools.fs_compound_db*), 64
 fitEOS () (*debyetools.potentials.BM method*), 30
 fitEOS () (*debyetools.potentials.EAM method*), 32
 fitEOS () (*debyetools.potentials.MP method*), 31
- ## G
- gen_Ps () (in module *debyetools.aux_functions*), 39
 gen_Ts () (in module *debyetools.aux_functions*), 39

I

`intAnharmonicity` (class in *debyetools.anharmonicity*), 47

L

`load_cell()` (in module *debyetools.aux_functions*), 41

`load_doscar()` (in module *debyetools.aux_functions*), 41

`load_EM()` (in module *debyetools.aux_functions*), 40

`load_V_E()` (in module *debyetools.aux_functions*), 40

M

`MG` (class in *debyetools.potentials*), 31

`min_G()` (*debyetools.ndeb.nDeb* method), 37

module

debyetools.anharmonicity, 43

debyetools.aux_functions, 39

debyetools.defects, 51

debyetools.electronic, 54

debyetools.fs_compound_db, 64

debyetools.ndeb, 35

debyetools.pairanalysis, 66

debyetools.poisson, 33

debyetools.vibrational, 61

debyetools.XS, 59

`MP` (class in *debyetools.potentials*), 31

`MU` (class in *debyetools.potentials*), 31

N

`N_at_Fermi()` (in module *debyetools.electronic*), 57

`nDeb` (class in *debyetools.ndeb*), 35

`neighbor_list()` (in module *debyetools.pairanalysis*), 66

`NfV()` (*debyetools.electronic.Electronic* method), 54

`NfV2m()` (in module *debyetools.electronic*), 58

`NfV_poly_fun()` (in module *debyetools.electronic*), 58

P

`pair_analysis()` (in module *debyetools.pairanalysis*), 66

`poisson_ratio()` (in module *debyetools.poisson*), 33

`PT` (class in *debyetools.potentials*), 31

Q

`quiet_pa()` (in module *debyetools.poisson*), 34

R

`RV` (class in *debyetools.potentials*), 31

S

`S()` (*debyetools.anharmonicity.Anharmonicity* method), 44

`S()` (*debyetools.defects.Defects* method), 51

`S()` (*debyetools.electronic.Electronic* method), 54

`S()` (*debyetools.XS.Xs* method), 60

`set_int_anh()` (*debyetools.vibrational.Vibrational* method), 62

`set_theta()` (*debyetools.vibrational.Vibrational* method), 62

`Svac()` (*debyetools.defects.Defects* method), 51

T

`TB` (class in *debyetools.potentials*), 31

V

`Vibrational` (class in *debyetools.vibrational*), 61

X

`Xs` (class in *debyetools.XS*), 59